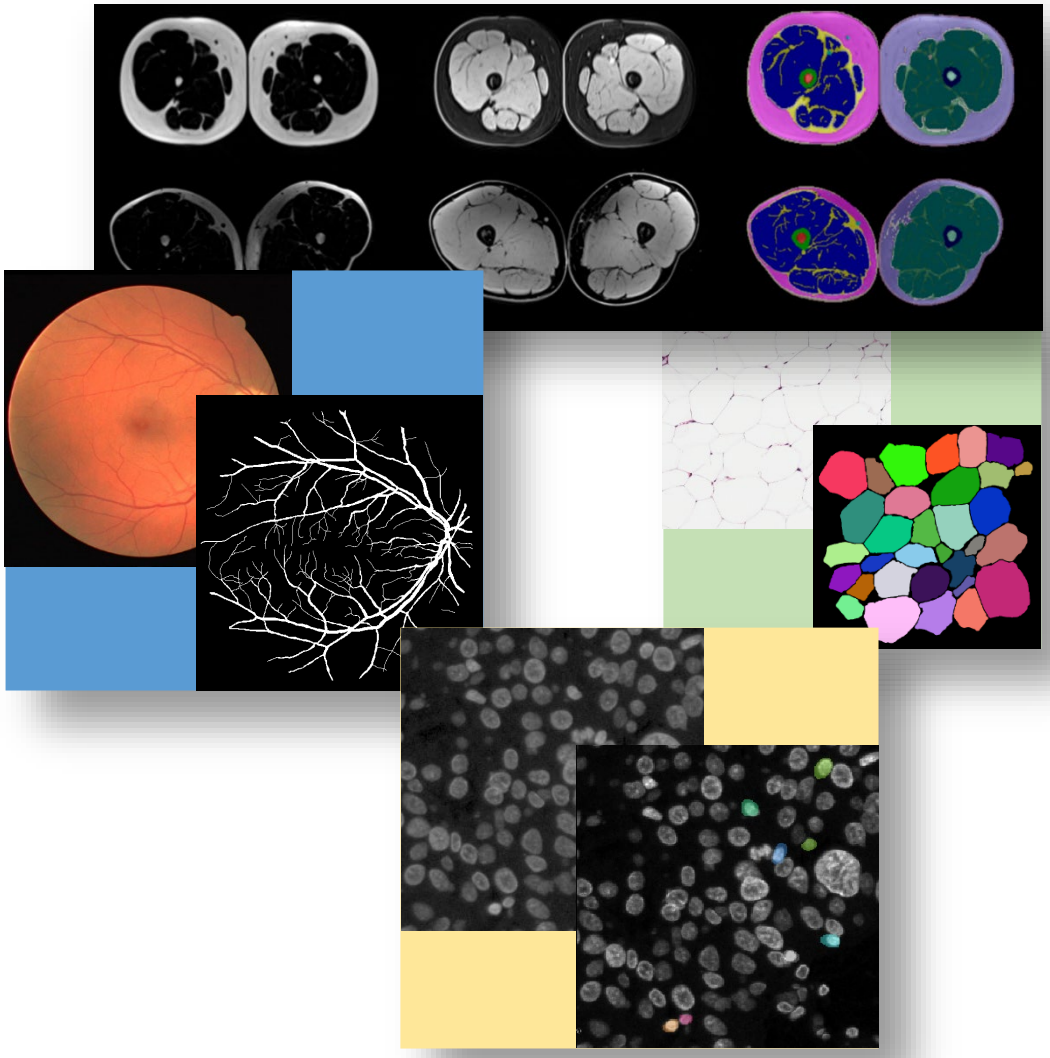
Lab Sessions

Digital Image Processing



Arantzazu Villanueva Larre

Mikel Ariz Galilea

Rafael Cabeza Laguna

November, 2024

Version 20241118

TABLE OF CONTENTS

LAB SESSION 0 INTRODUCTION 4

1. INSTALLING PYTHON 4
2. FIRST STEPS 4
3. MAIN DATA TYPES 7
4. OBJECT ORIENTED PROGRAMMING 8
5. BASIC STATEMENTS: `WHILE`, `IF`, `FOR`… 9
6. FUNCTIONS 11
7. ENVIRONMENTS 13
8. OPENCV 16
9. SETTING UP OUR WORKING VIRTUAL ENVIRONMENT 17

LAB SESSION 1 FIRST OPERATIONS WITH IMAGES 20

1. PRELIMINARIES 20
2. FIRST STEPS: LOAD, VISUALIZE AND SAVE AN IMAGE 21
3. EXPLORING THE IMAGE 22
4. PLAYING WITH COLORS 24
5. ADDING AND BLENDING IMAGES 24
6. MEDICAL IMAGES 27

LAB SESSION 2 IMAGE ENHANCEMENT AND FILTERING 32

1. PRELIMINARIES 32
2. CALCULATE IMAGE HISTOGRAM 33
3. CONTRAST STRETCHING 36
4. SPATIAL CONVOLUTION 37
5. DISCRETE FOURIER TRANSFORM (DFT) 39

LAB SESSION 3 MORPHOLOGY 44

1. PRELIMINARIES 44
2. BASIC OPERATIONS: EROSION, DILATION, OPEN AND CLOSE 45
3. ADVANCED OPERATIONS 47
4. EXTENSION TO GRAY LEVEL 50
5. EXTRACTION CONNECTED COMPONENTS 51
6. ADVANCED EXERCISE: CONVEX HULL 52

LAB SESSION 4 SEGMENTATION 53

1. PRELIMINARIES 53
2. BASIC THRESHOLDING 54
3. ADAPTIVE THRESHOLDING 55
4. CANNY EDGE DETECTION 57
5. HOUGH TRANSFORM 58
6. WATERSHED TRANSFORM 60
7. ADVANCED EXERCISE: PRINCIPAL COMPONENT ANALYSIS 63

LAB SESSION 5 DEEP LEARNING 66

LAB SESSION P1 PROJECT: PRELIMINARIES 67

1. CONSTRUCTING A PYTHON LIBRARY 67
2. DATA STRUCTURES IN PYTHON 67
3. ADVANCED INDEXING 68
4. MEASURING EXECUTION TIME 68
5. EXTRACTING CHARACTERISTICS FROM BLOBS 69
6. MEASURING ACCURACY 70

LAB SESSIONS P2-P5 SEGMENTATION OF CD8+ LYMPHOCYTES FROM MICROSCOPY IMAGES OF UROTHELIAL CARCINOMA 73

1. INTRODUCTION 73

CLINICAL FRAMEWORK 73

IMAGE ACQUISITION 75

2. OBJECTIVES 75
3. DATASET 76
4. DELIVERABLES 77
5. MATERIALS 82
6. GRADING POLICY 82
7. IMPORTANT ISSUES 83
8. REFERENCES 83

ANNEX I WORKING WITH ENVIRONMENTS 84

LAB SESSION 0
INTRODUCTION

In order to follow the lab sessions in this class you should get familiar with some new programming tools, i.e. Python and Visual Studio Code. Additionally, you would need to rely on several libraries being OpenCV the most important one.

Let's go little by little…

Python is a powerful programming language, easy to learn. It is an object-oriented programming tool using a simple approach.

1. Installing Python

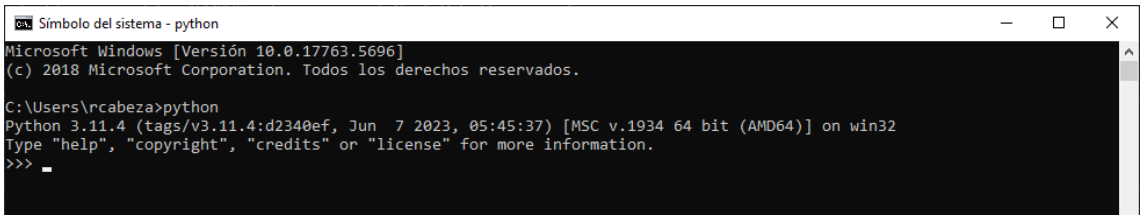
Install Python for Windows (in the lab computers the installation will be ready to use). You should follow these steps if you want to have the programming framework in your personal computer, or alternatively if you want to learn how to install Python in a general situation. Our first step is the installation of the base package: Python 3.11.4 (64 bits). You can download it from:

https://www.python.org/ftp/python/3.11.4/python-3.11.4-amd64.exe

https://www.python.org/ftp/python/3.11.4/python-3.11.4-macos11.pkg

IMPORTANT	*During the installation, check "Add Python 3.11 to PATH". In this manner, it will be included in system path*

Open windows command prompt and execute `python`

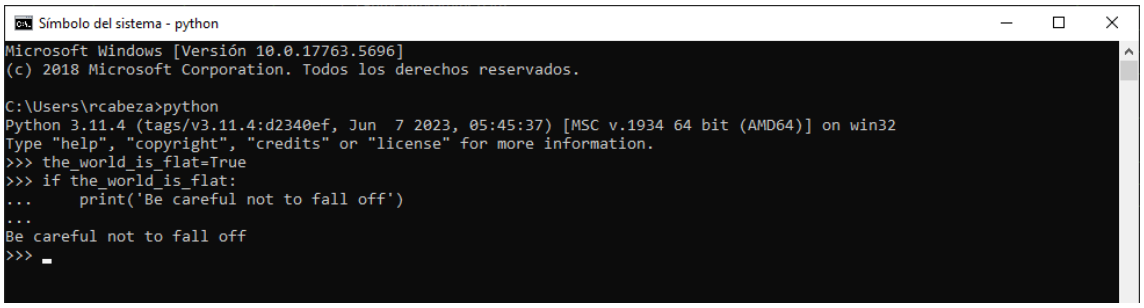


Now you are working in Python. Execute `2+2`.

To return back to the windows command prompt type `quit()`.

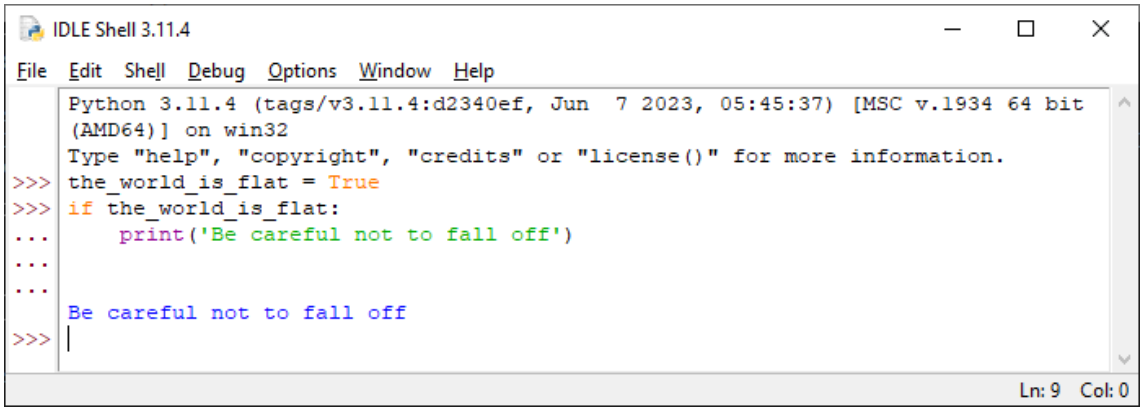
2. First steps

To follow copy the following piece of code even if you don't know very well the syntax details.



Python has its own syntax to operate with number, lists, matrices etc. Moreover, as in any other language you can create `for`, `if`… statements but of course, you need to learn how to use them… Please, note the indent of the line below the `if` command. In Python, indents are critical to write proper code.

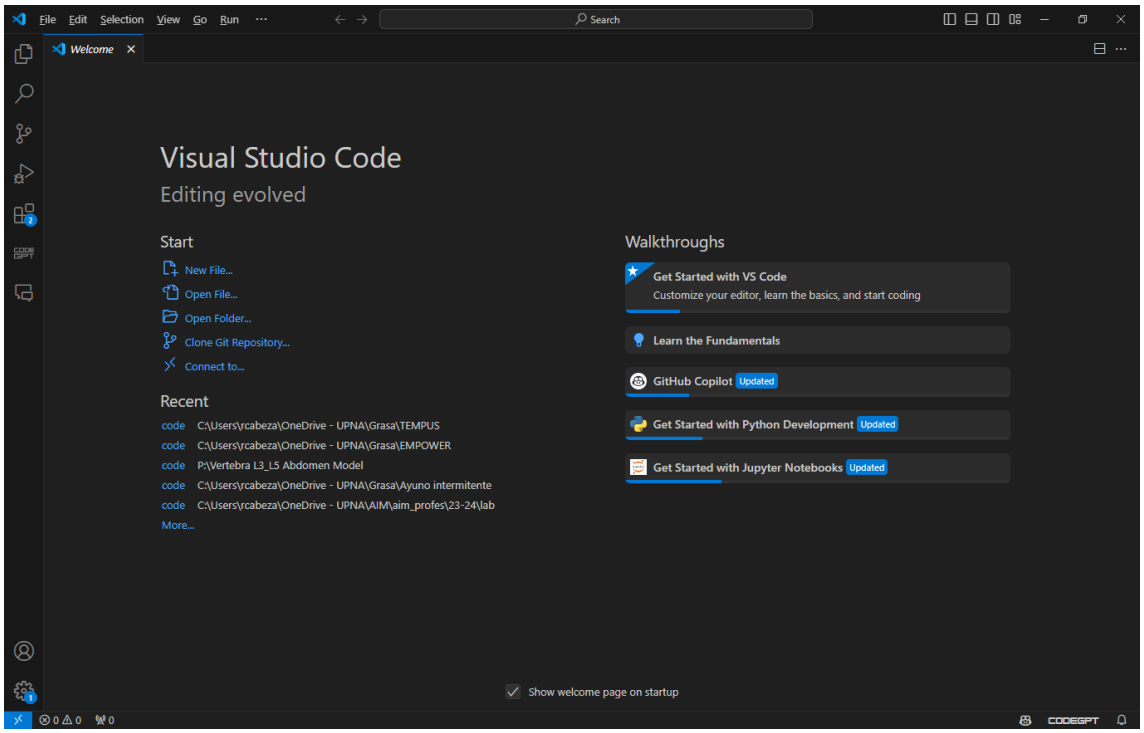
Windows command prompt is not a friendly code editing interface… There are several ways to run Python code more easily. Python installation itself provides a code editing software called IDLE. You can launch IDLE module typing `python -m idlelib` into the command window.



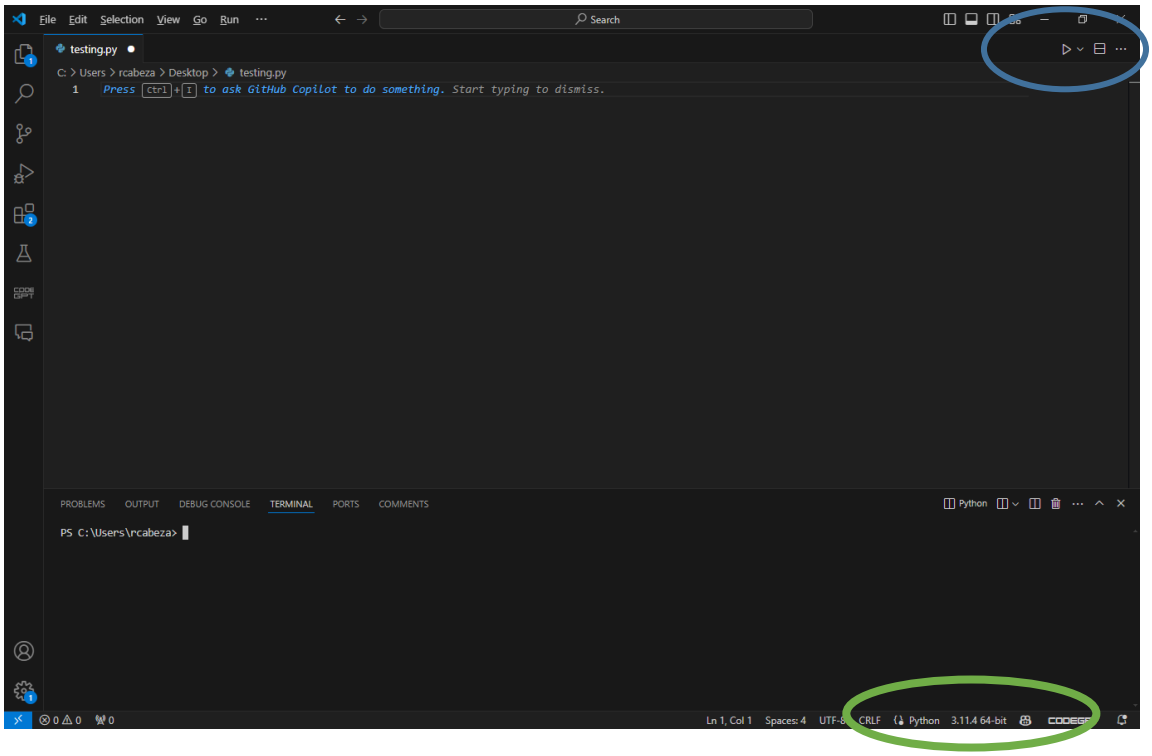
In our lab sessions, Visual Studio Code (VSC) is going to be used as the main code editing software. Among the existing options, we have selected Visual Studio Code for the lab sessions. It is a powerful code editing software optimized for coding, debugging and running available for Windows, macOS and Linux. It supports languages such as JavaScript, TypeScript and Node.js and has a rich ecosystem of extensions for other languages (such as C++, C#, Java, Python, PHP, Go) and runtimes (such as .NET and Unity).

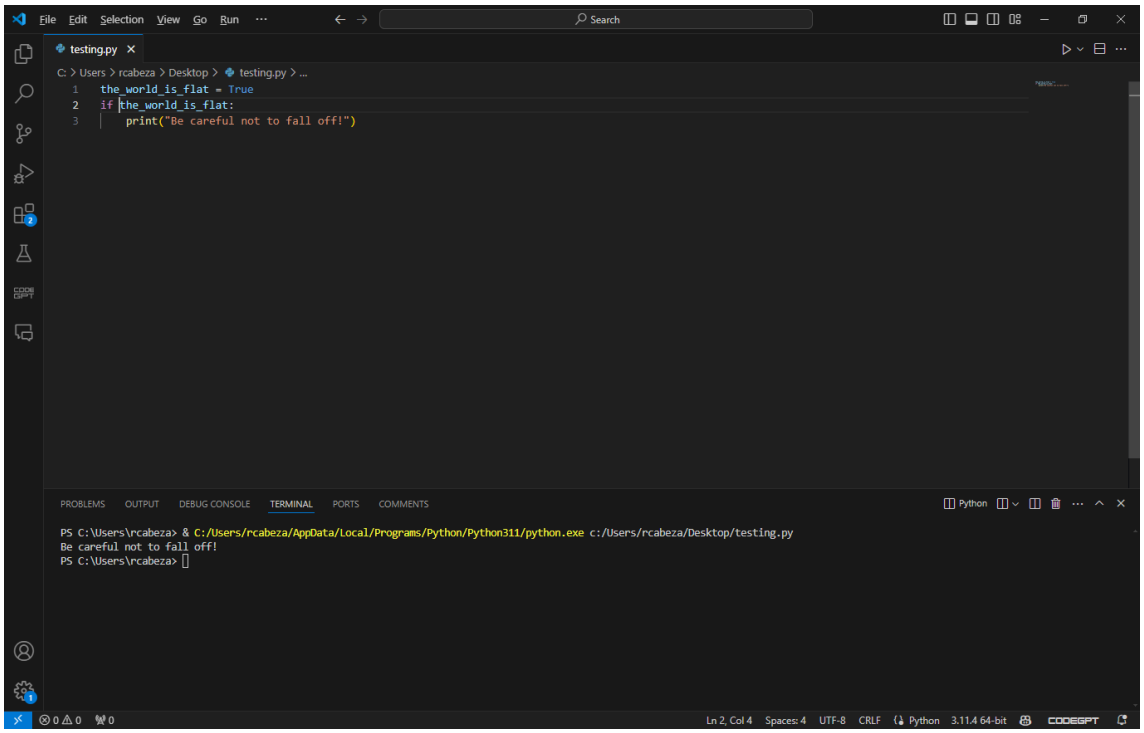
You can install VSC from here:

https://code.visualstudio.com/download



Create a new file testing.py. By adding '.py' extension to your file, VSC understands that Python is going to be used. Note that, in the lower bar of your window, "Python X.X.X" is shown and a play icon shows up in the right area of the window.

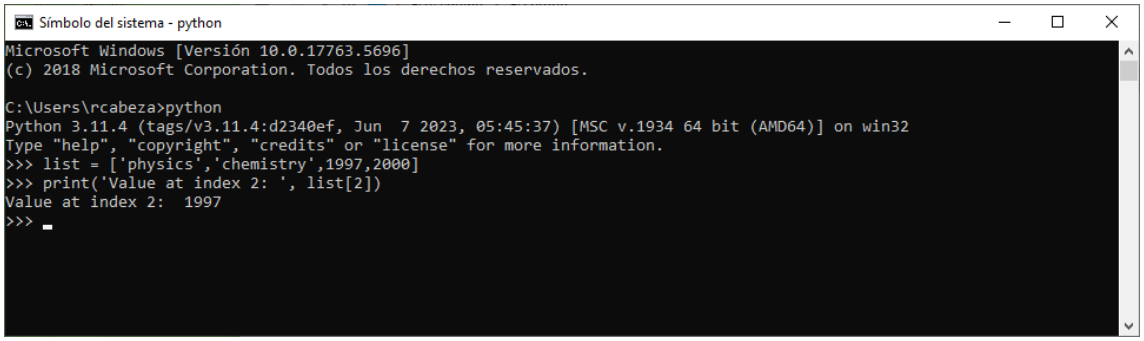




The rest of the examples in this document are shown using windows command prompt. Again, this is not the friendliest coding interface. You can use cmd, IDLE or VSC for replicating the examples. Remember that VSC will be the software we will use in the lab sessions.

3. Main data types

Lists are the most versatile data type in Python



Have a look and copy the above example… what do you conclude about indexing? Does Python start indexing at 0 or 1?

More about numbers:

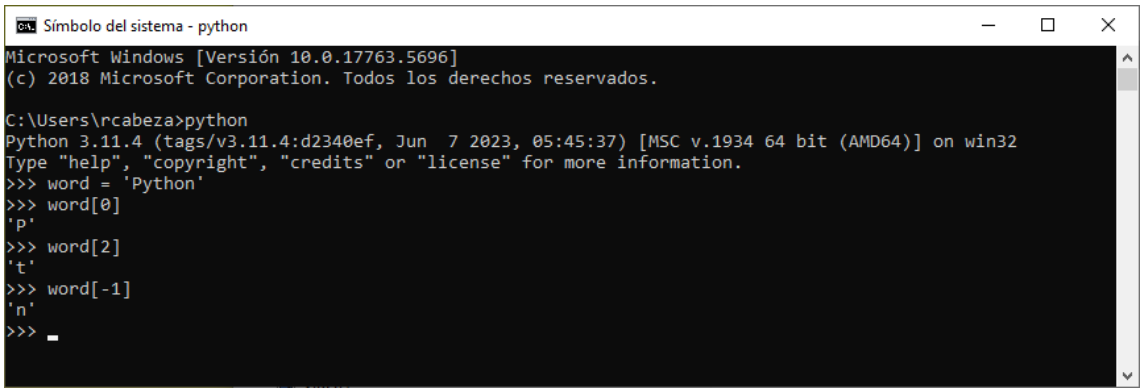
https://docs.python.org/3/tutorial/introduction.html#numbers

More about lists

https://docs.python.org/3/tutorial/introduction.html#lists

We encourage you to follow the links thoroughly… try to replicate and understand the examples included in the tutorials.

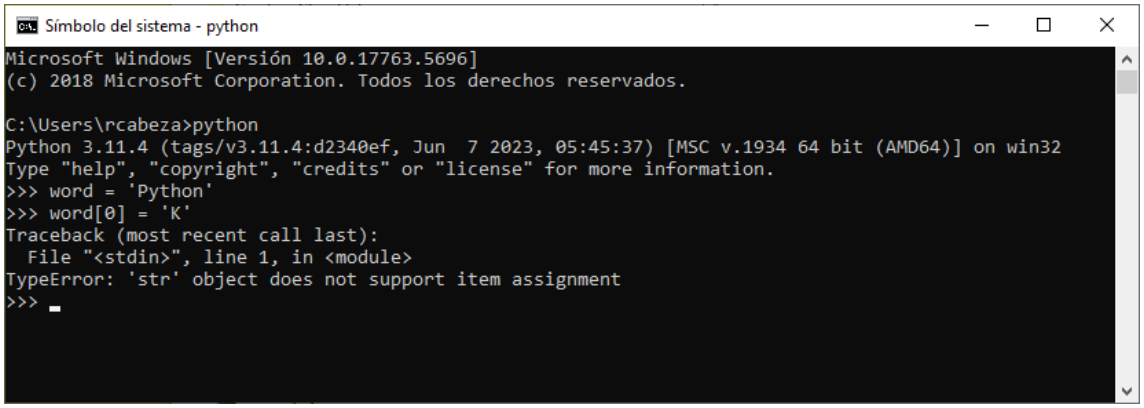
Python can also manipulate strings:



Negative indexing starts from the end of the string. Strings can be also indexed to obtain a part of the word or, as expected, concatenate it with other letters by using +:



IMPORTANT	*Python strings cannot be changed — they are immutable. Therefore, assigning a new value to an indexed position in the string results in an error.*



More about strings:

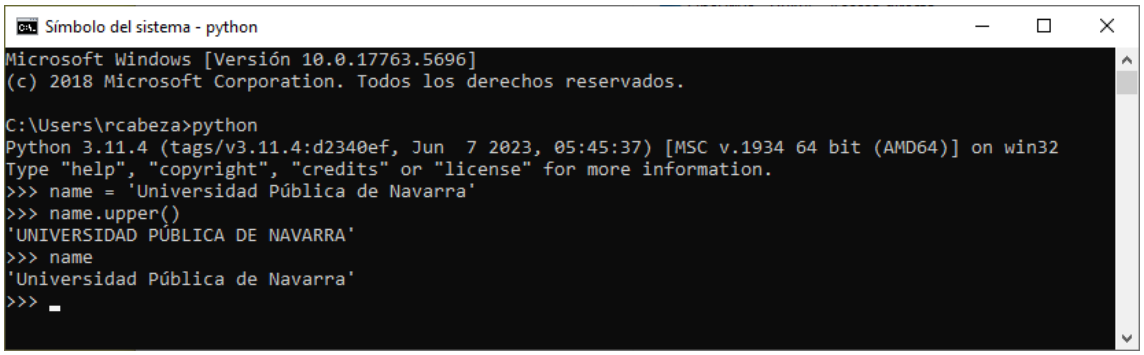
https://docs.python.org/3/tutorial/introduction.html#strings

4. Object oriented programming

Object oriented programming is a programing paradigm constructed around “objects”. Each object encapsulates data and methods that can manipulate/modify the data in the object.

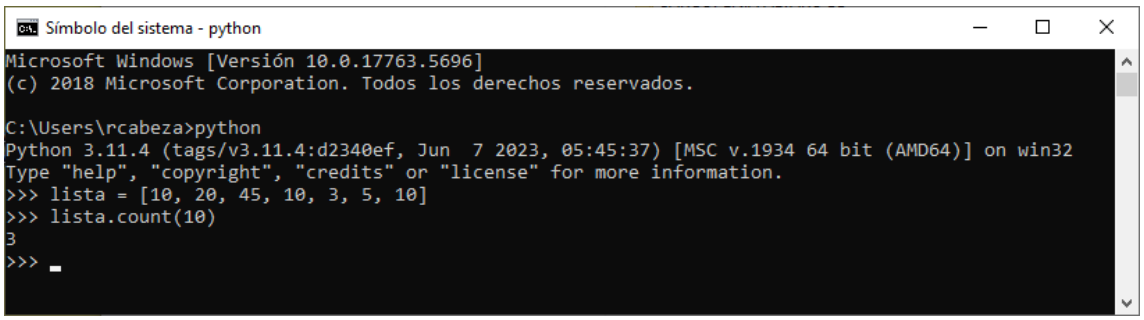
Having a variable of a specific data type, e.g. string, list… the standard syntax to access a method is using ..

In the following example, a variable of type string is defined. The method .upper() retrieves the upper letter case version of the string.



https://docs.python.org/3/library/stdtypes.html#string-methods

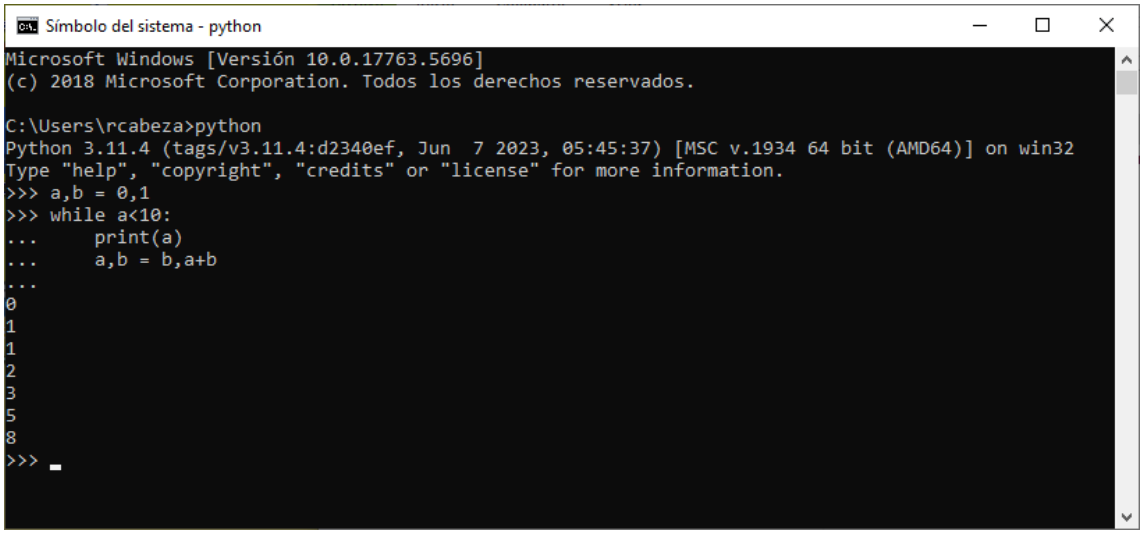
Let's do another example using lists:



5. Basic statements: `while`, `if`, `for`…

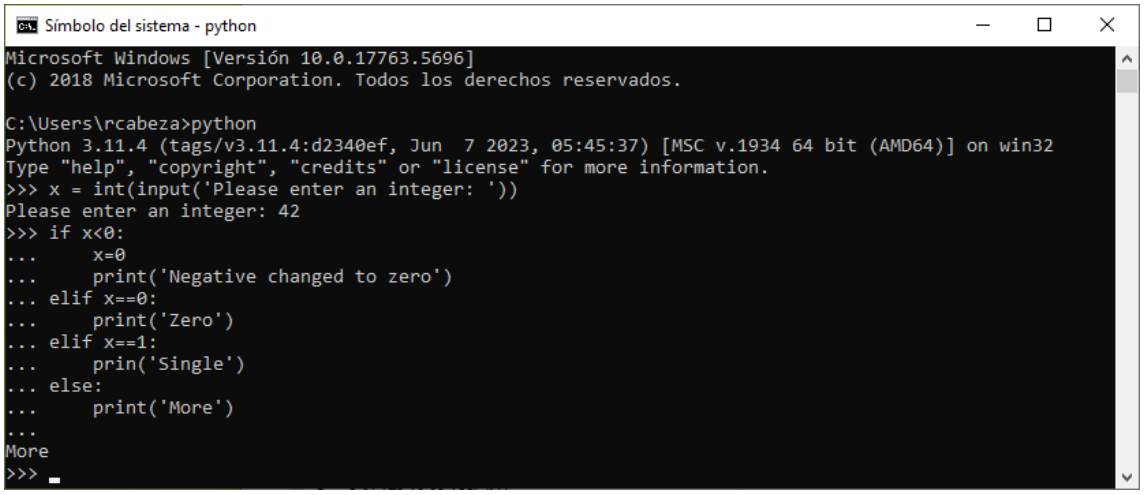
Python as in most of the programing languages provides its own syntax for the most common use statements.

a) `while`



Note the following main aspects: first, `:` is required after the condition; second, indentation is required for all the instructions included in the loop. These two elements are repeated for most of the statements and definitions requiring more than one line of code (`for`, `if`, functions…)

b) `if`

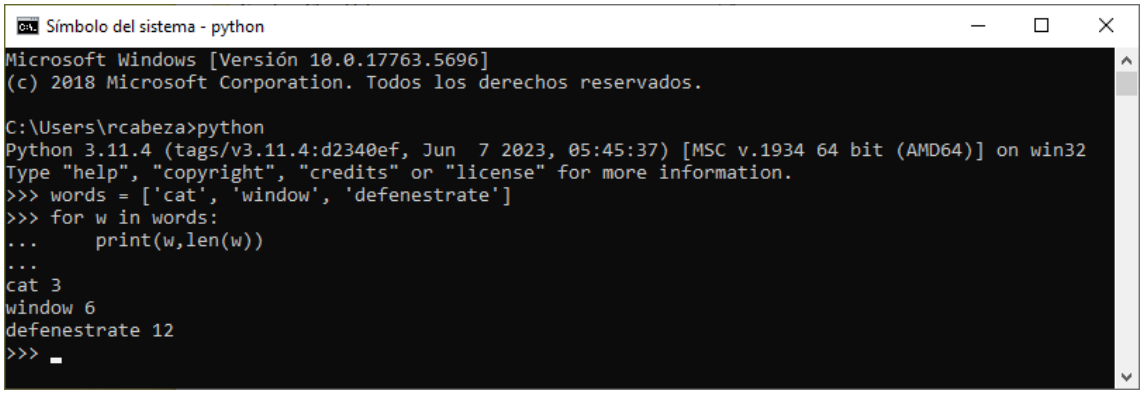


c) `for`



Note that we are running a for loop from 0 to 4 when we write `i in range(5)`

The command `for` provides alternatives for indexing.



Please have a look to the following links about alternative statements and indexing and replicate the examples.

https://docs.python.org/3/tutorial/controlflow.html#break-and-continue-statements-and-else-clauses-on-loops

https://docs.python.org/3/tutorial/controlflow.html#pass-statements

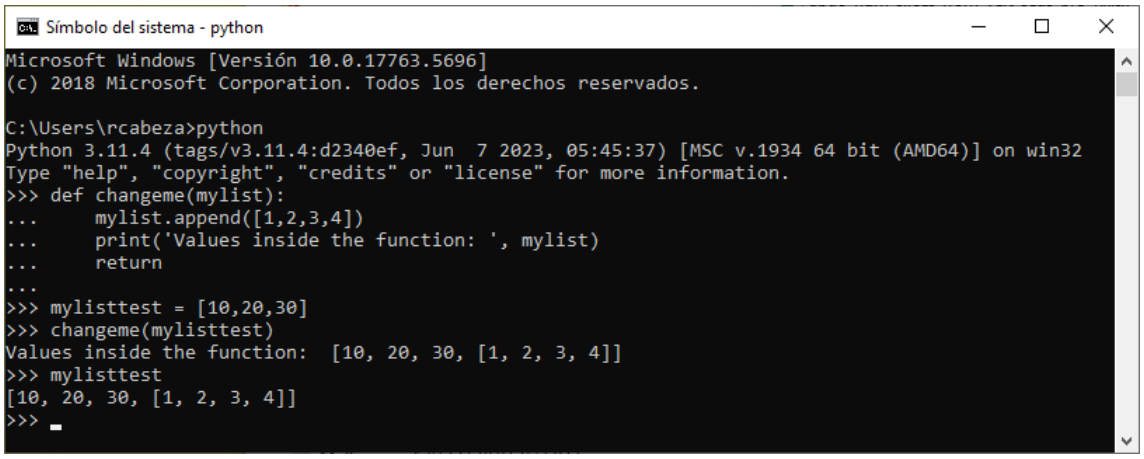
6. Functions

In programming, a function is a sequence of instructions forming a reusable code called under a single command line. In order to define a function `def` is used in python.

```
def functionname( parameters ):
    "function_docstring"
    function_suite
    return [expression]
```

Again note the requirement of : after the name of the function and the indentation for the instructions inside the function.

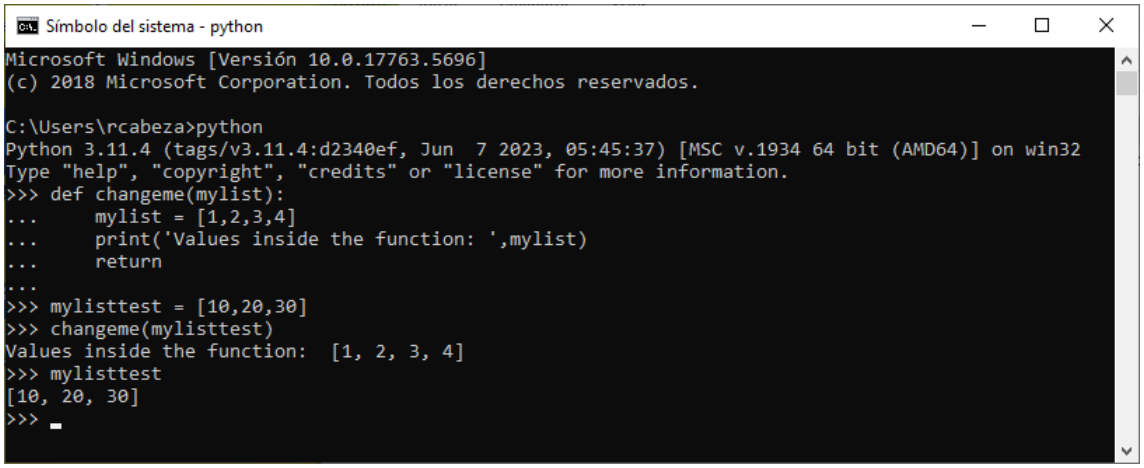
In the following example, a function without output arguments is shown. Instead, the “results” of the function are printed out.



Here, we are maintaining reference of the passed object and appending values in the same object. Remember that Python supports object oriented programming language. Each data type

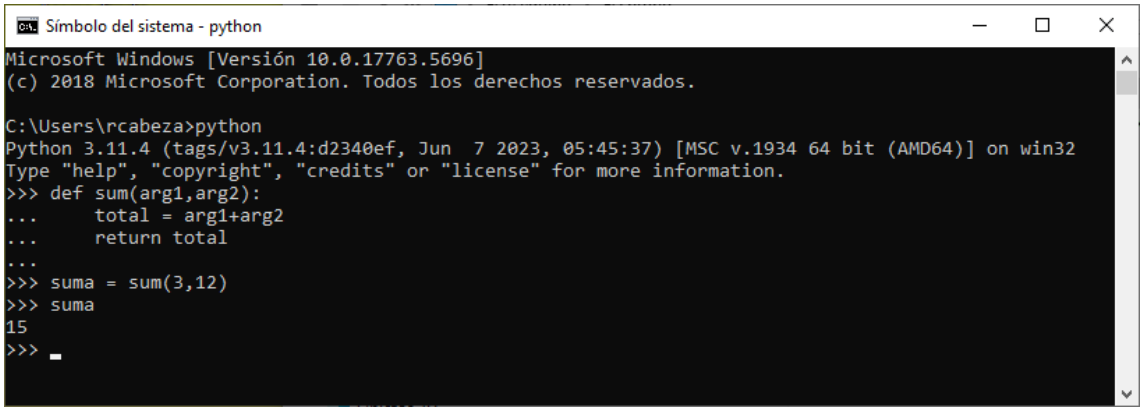
has its own methods. In this case, `.append` is a method of data type `list` that adds elements to the list.

Note the difference with respect to the following function

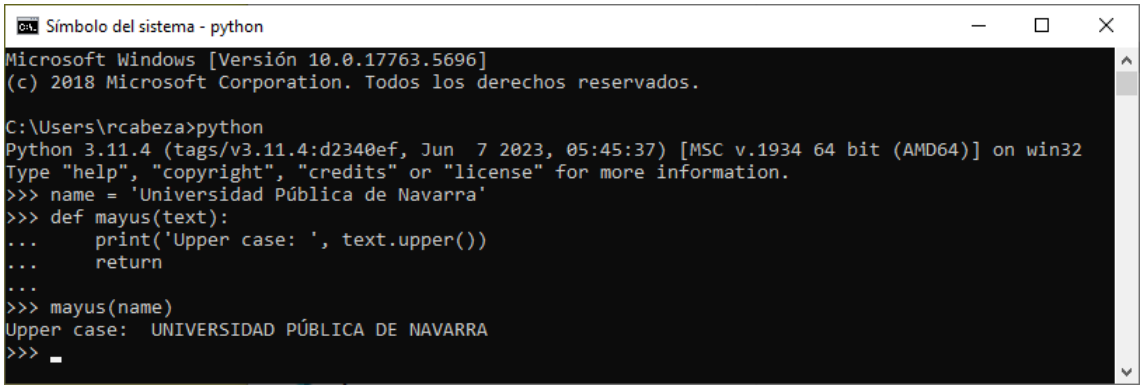


In this case, the reference of the passed object is lost and a new reference is given to `mylist` inside the function. The variable `mylisttest` is not modified inside the function and keeps its value.

The examples above provide the output printed out in the screen however; many times, we are interested in saving the result of a function in a variable. To this end, `return` is used.



Another example:



More about functions:

https://docs.python.org/3/tutorial/controlflow.html#defining-functions

We thoroughly recommend you to follow the following tutorial (sections 1 to 4):

https://docs.python.org/3/tutorial/

IMPORTANT	*It is important to follow a standard coding convention. Here you can find very useful style guidelines. Please, read it carefully*

The above lines are a too simple introduction to Python and you would need a more intensive training to follow lab sessions.

7. Environments

The standard installation of Python provides basic data types, functions and methods. However, for many applications based on Python more complex data types and functions are required. The way to include all the power of Python in the basic installation is to encapsulate specific features in independent libraries that can be installed additionally as required. In this manner, when programming explicit libraries can be used if needed for your application purposes.

The amount of libraries available for Python is huge with varied objectives. You can find libraries for array manipulation, image processing, graphics, deep learning… The libraries, as is Python, are continuously updated and new versions show up constantly. This frequent library updating process provides new functionalities to the libraries but many times causes problems in our programing tasks since the new version of the specific library is not compatible any more with the rest of our libraries already in our project.

In order to overcome this issue, Python provides the virtual environment functionality.

A Python virtual environment is a separate framework inside Python that helps to keep dependencies between Python and the employed libraries versions in a kind of an isolated Python setup.

Imagine that for project A you need the following versions of python and libraries: python 3.6, numpy 1.14.0 and scikitlearn 0.2

On the contrary, for project B you need the following versions of python and libraries: python 3.11.4, numpy 1.14.0 and scikitlearn 0.23

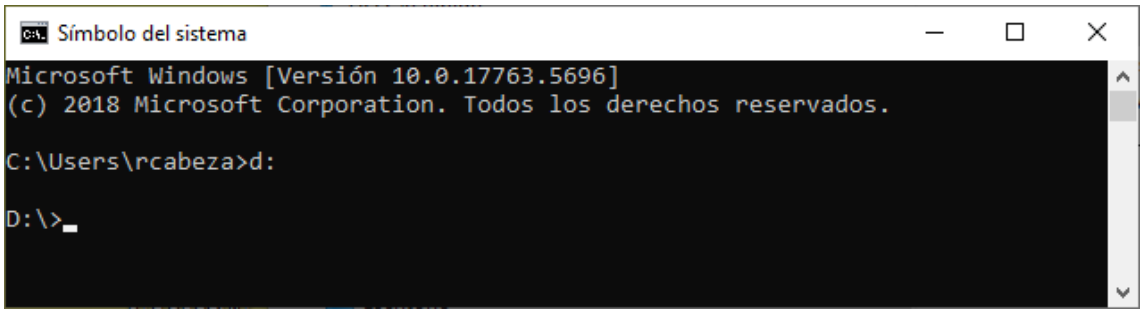
If I want to have both projects running in the same computer how can I ask the computer to use different library versions for the different projects?... by using virtual environment tools. In each virtual environment, you install the specific versions of python and libraries needed. A self-contained directory tree contains a Python installation for a particular version of Python, plus a number of additional packages/libraries. In this manner, we would create two separate virtual environments in two separate directories, i.e. one for project A and the other one for project B. Once the environments are created, the corresponding libraries would be installed.

In order to create a virtual environment, a separate folder is to be created in our directory tree. Create "*PDI/projectA*" in your computer.

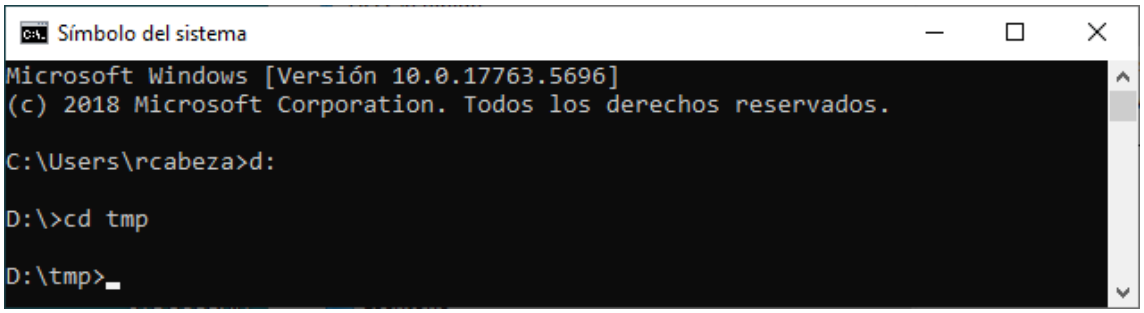
IMPORTANT	*We recommend you to use CMD for the setting up of virtual environments and package/libraries installation.*

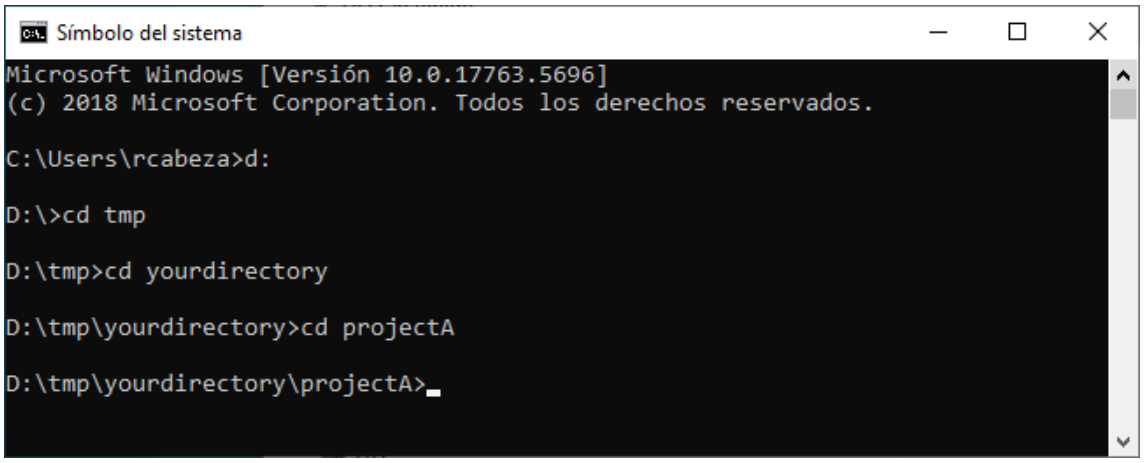
Using the command window prompt, go to the created directory.

If you need to change the disk, for example from C: to D: type `d:`

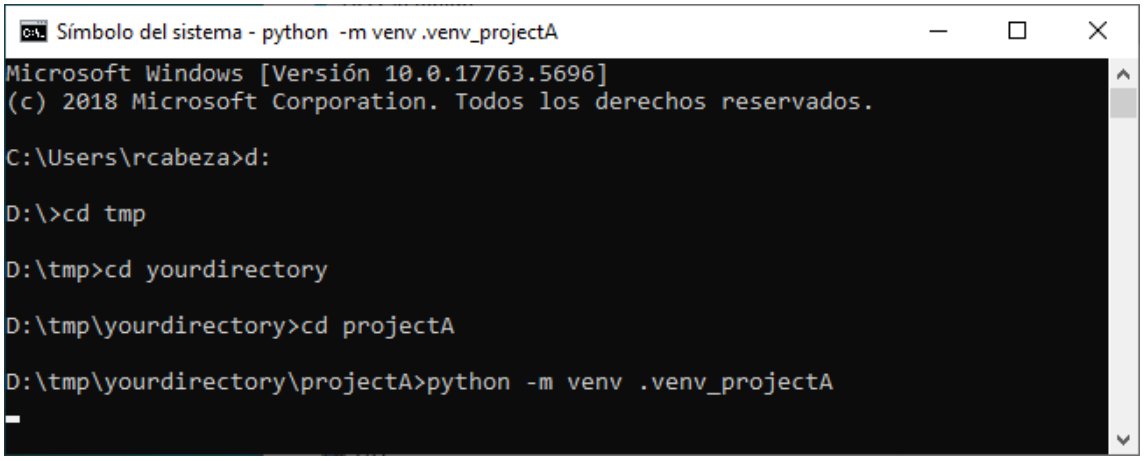


Using `cd` you can navigate through your folders:





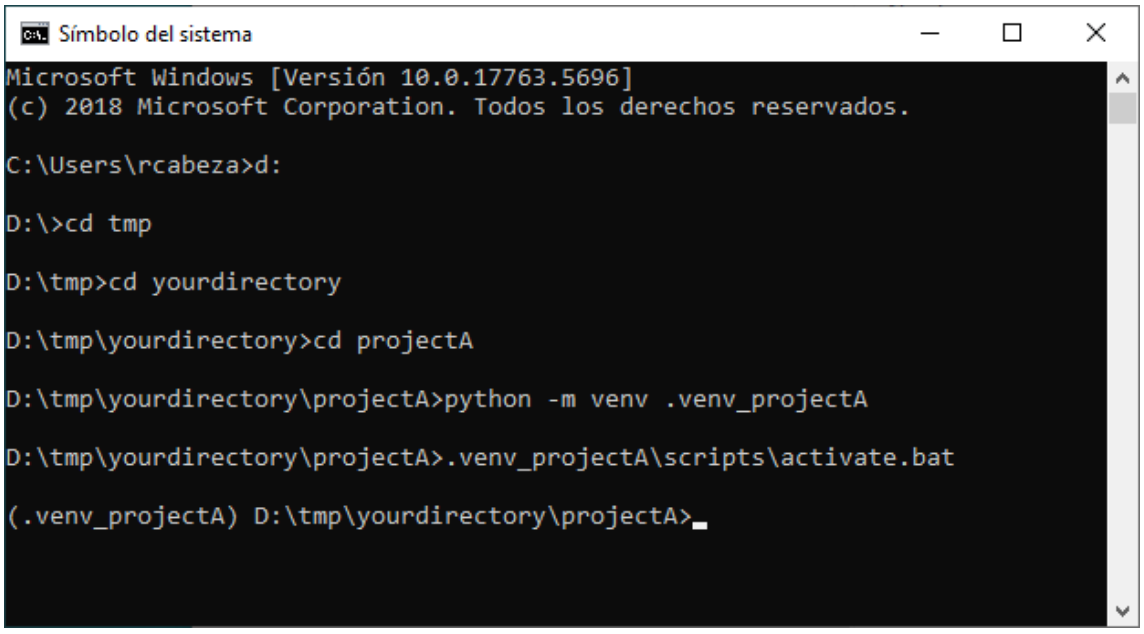
Once you are in the folder, create the virtual environment named as: .venv_projectA



	*On Mac, use:* `python3 -m venv venv_projectA`

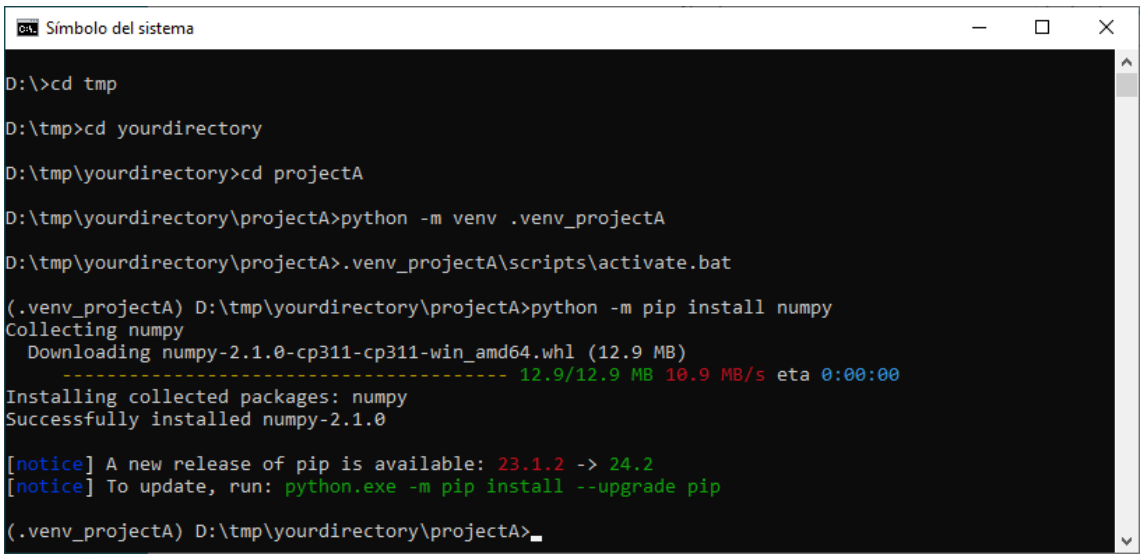
Once the environment is created, in your folder, a directory with the name of the environment should be found. Inside, a subfolder named as Scripts will show up in which a file activate.bat is set up.

By running `activate.bat` the environment is activated:



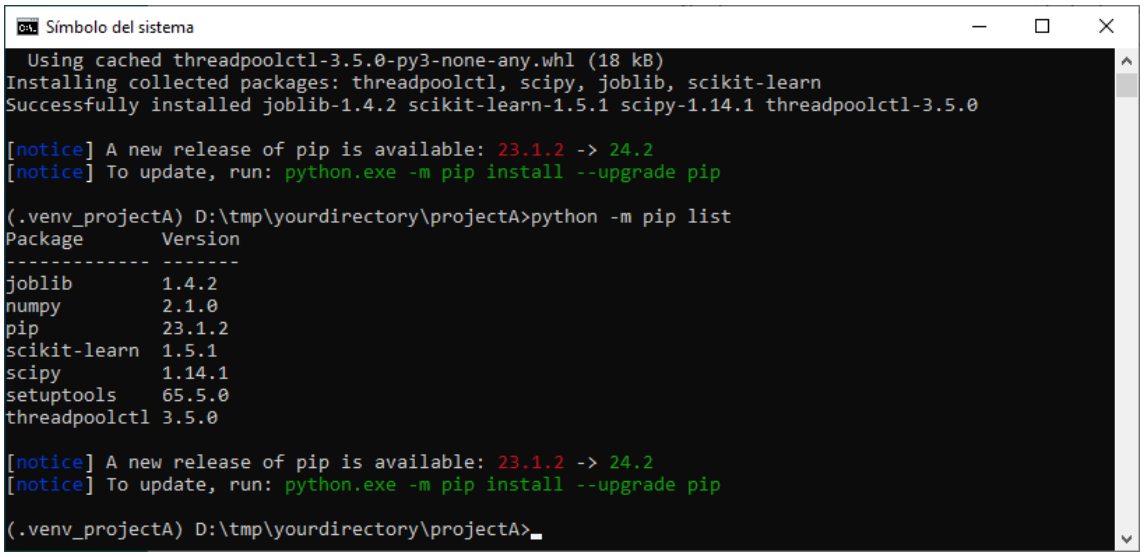
	On Mac, use: `source venv_projectA/bin/activate`

Now the libraries could be installed. The python command to install libraries is `pip`, e.g. `python -m pip install numpy` is the command to install numpy package.

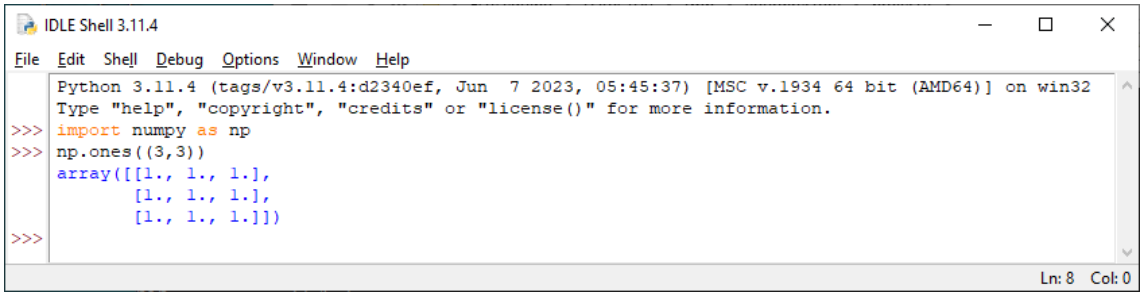


Install now scikit-learn package.

Using `pip list` the packages already installed in the virtual environment are shown.



Once the libraries are installed in your environment you should import in your code in order to be able to use them. In order to work with IDLE module under an specific environment, you have to launch this module (`python -m idlelib`), from the command window of the activated environment.



Setting up virtual environments sometimes requires time and is bothering to find the correct dependencies between libraries. Once you have an environment running you can "copy" the virtual environment created and "install" it in another machine. In this manner, exactly the same libraries and python version will be created and the working framework will be replicated avoiding potential version crashes. The command `python -m pip freeze > myenv.txt` is used to this end. A txt file is created that can be further installed in another machine with command `python -m pip install -r myenv.txt`

By means of `deactivate` the activation of the virtual environment disappears.

If you need to delete a virtual environment, it is enough to delete its corresponding folder.

More about environments and packages

https://docs.python.org/3/tutorial/venv.html

8. OpenCV

Image processing is a particular area of computer science and there are several python packages providing image analysis tools such as Scikit-image, OpenCV, Mahotas, SimpleITK, SciPy, Pillow, Matplotlib...

We have selected OpenCV as reference due to its wide possibilities. OpenCV supports a wide variety of programming languages such as C++, Python, Java, etc., and is available on different platforms including Windows, Linux, OS X, Android, and iOS. Interfaces for high-speed GPU operations based on CUDA and OpenCL are also under active development.

It is not necessary to install the full version of the OpenCV library in order to complete these lab sessions, but you can do it and learn with a wide and excellent set of examples included in the standard package.

OpenCV-Python is the Python API for OpenCV, combining the best qualities of the OpenCV C++ API and the Python language. Here you can find a complete tutorial of this API:

https://docs.opencv.org/master/d6/d00/tutorial_py_root.html

This specific API is the only part of the OpenCV that we'll need in our lab sessions. Its detailed configuration is described in the next lines.

9. Setting up our working virtual environment

In Annex I the procedure to create a virtual environment is summarized. All the points have been described in previous sections but it is highly recommended to read it. Please have a look to it in order to be sure you have perfectly understood all the steps.

In this last stage, we will create and setup the virtual environment for our lab sessions. The setup will be different depending on how you plan to work, i.e. using lab computers or your personal laptop.

Working with your personal laptop:

Create the working directory PDI/LAB/CODE in your selected directory.

Working with lab computers:

If you have a computer at home we recommend you to setup the framework before coming to the lab (if not, you could do it during the first lab session). It is highly recommendable to setup the environment in advance since it takes long to install everything.

The advice is to create the environment in a USB memory device, i.e. your working directory is going to be contained in the USB key so that you can bring it to the lab. You don't need to copy the USB contents into the lab computers, instead, you will work in the USB key during the lab session.

Create the working directory PDI/LAB/CODE in your selected directory in your computer or in the USB key.

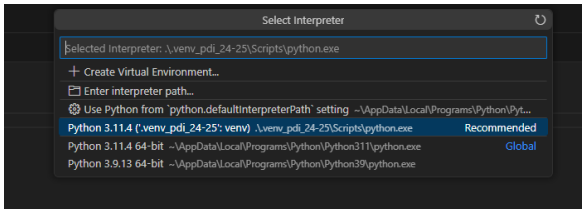
The environment to be created is *.venv_pdi_24-25* in the working directory.

```
python -m venv .venv_pdi_24-25
```

Once you create it, proceed to activate it. As part of the supplementary material supplied with this lab session a *requirements.txt* is provided with the required library settings. Install it as explained in section 7 in your created environment.

The main libraries installed in your environment by installing requirements.txt are: numpy, matplotlib and opencv-contrib-python.

A very last step is left before moving to the next sessions. When opening VSC the code you write is interpreted by the basic python installation Python 3.11.4 64 bits. In order to work in a selected virtual environment once you are in the working directory (PDI/LAB/CODE), create a new file with the '.py' extension, e.g. test.py, and use "Python: Select Interpreter" command from the Command Palette (`Ctrl+Shift+P`)

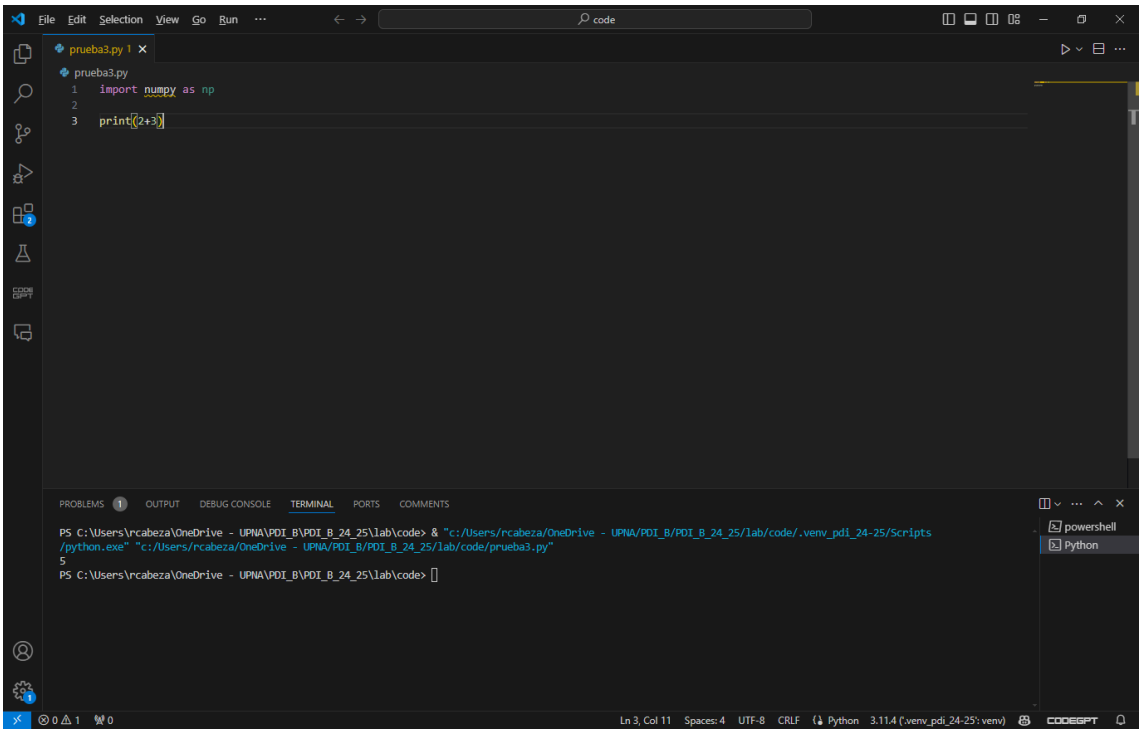


Alternatively, as a shortcut of the above action, you can double click in the lower bar of the main window where python version is shown.

The Python: Select Interpreter command displays a list of available global environments, conda environments, and virtual environments. Select your environment (*.venv_pdi_24-25*). If you don't have it available in the list box click "Enter interpreter path…" go to .venv_pdi_24-25/Scripts and select python.exe. You can find more information here.

	*On Mac, before create the test.py, go to File menu, click on Open Folder… and select the folder with your code*

Then you can start coding and importing the required libraries. The files created during lab sessions are to be saved in the same working directory (PDI/LAB/CODE).



Let's check that your environment is working…

In your test.py file import OpenCV `import cv2 as cv`

Copy any image you want into your folder, PDI/LAB/CODE, and read it with the function `cv.imread`

Run it!

```
import cv2 as cv

img = cv.imread('myimage_path')
cv.imshow('My window name', img)
cv.waitKey(0)
```

If you get an error, follow these steps:

- Check that the value of the `myimage_path` is correct
- If the library cannot be imported check that you are working in your environment. You need to select the correct environment, i.e. *.venv_pdi_24-25* (see above)
- If the library cannot be imported and your environment is correct… you should review it. Go to **cmd** activate the environment and check that the libraries have been correctly installed with the command: `pip list`

A nice tutorial about VSC can be found here

https://code.visualstudio.com/docs/python/editing

Finally, some highly interesting and advisable tutorials are provided here. Short descriptions are provided about the different sources.

PYTHON

https://docs.python.org/3/tutorial/ The best site to read, consult and learn Python. A lot of examples are provided and the presentation sequence is very appropriate.

https://www.tutorialspoint.com/python/python_lists.htm A good alternative. Well focused and structured.

https://www.w3schools.com/python/ Not much detailed but it is a quick and Handy consulting document.

https://www.datacamp.com/ An excellent site for exploration. Many courses available.

VISUAL STUDIO CODE

https://code.visualstudio.com/docs/python/python-tutorial For setting up and starting with VSC

To follow some more advanced sites for Python edition using VSC:

https://code.visualstudio.com/docs/python/editing,

https://code.visualstudio.com/docs/python/linting,

https://code.visualstudio.com/docs/python/debugging.

https://docs.opencv.org/master/d9/df8/tutorial_root.html

LAB SESSION 1
FIRST OPERATIONS WITH IMAGES

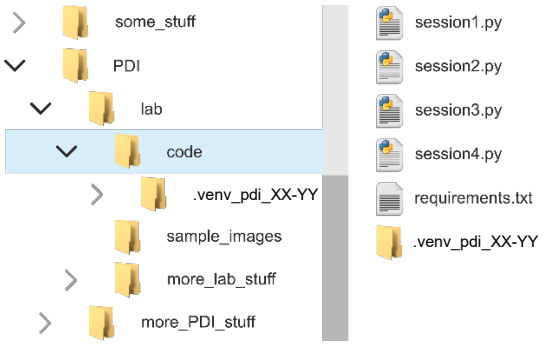
1. Preliminaries

We recommend you employ a single file for the whole session. In this manner, you can import the required libraries in the heading of the file and you don't need to load the libraries for every single file created during the lab session. Remember that you can comment sections of code to focus in the alternative exercises provided.

Open VSC and activate the *.venv_pdi_24-25* environment. It is assumed that you have done session 0 successfully and that you are familiar with the setting up procedure of the environment. If not, go back to session 0.

TIP	*Remember that all the images required for the session are contained in sample_images folder that you need to download in your computer.*
TIP	*For those of you working with lab computers, remind to use the environment contained in the USB key.*

So, the final folder tree in your computer could look like this:



Please, double check that the structure of the folder in your computer follow the above scheme.

TIP	*It is convenient to change the working path to your current working directory. In this manner, relative paths can be used and the code can be more easily exported to other machines.*

```
import os
#Getting the path for this file
project_path = os.path.dirname(__file__)
#Changing out current working directory. In this way, we can work with
relative paths
os.chdir(project_path)
```

Note that `__file__` is a predefined variable returning the path of our script. The variables or methods defined with **two** underscores on each end are usually named as a *dunder*. You can consult this link to get a detailed explanation about using underscores in Python.

 TIP	*You can comment lines of code using #. When you need to comment several lines of code use ''' at the beginning and end of the commented code.*

2. First steps: load, visualize and save an image

Libraries: *cv2, sys*

```
import cv2 as cv
from matplotlib import pyplot as plt
import sys
import os
```

Have a **quick** look to the description of the libraries to get an idea about their potential contents.

Load an image

Using OpenCV you can load an image.

Load *starry_night.jpg* image

```
starry_img = cv.imread(pathtotheimage)
```

Remember that it is convenient to use relative paths. So the input argument to `cv.imread()` function could look like `'../sample_images/starry_night.jpg'`.

Your code should consider the case in which the image is not found. If that is the case, the result provided by `cv.imread()` is *None*. Then an error message is to be retrieved and the program finished.

```
sys.exit('Could not read the image')
```

Using an `if` statement include this case after reading the image.

Visualize an image

Once the image is loaded visualize it

```
cv.imshow('title of the window',starry_img)
```

The window closes too fast….

In order to keep the window opened OpenCV provides the following solution

```
k = cv.waitKey(0)
```

The window remains open for the number of milliseconds. If the number of milliseconds is 0, the execution is interrupted until a key of the keyboard is pressed. The variable `k` contains the Unicode of the pressed key.

 **TIP**	*The first 128 elements of Unicode are the same as ASCII.*
 **TIP**	*In order to use relative paths remember that './' means the actual working folder (typically the folder of your script or project), while '../' denotes that your folder of interest is not contained in the actual folder but in its parent directory.*
 **TIP**	`k = cv.waitKey()` *will keep the windows opened until the execution is finished. However, if* `cv.waitKey()` *is used the windows will remain open even if the code is finished.*

Another option for displaying images is by using the `imshow()` function from the Matplotlib library. This second option has several advantages over its twin function in the OpenCV library. Firstly, it allows you to visualize the indexes as well as the value of each pixel. You can also easily zoom in on a specific area of the image. The display window can be resized directly. Another difference is that it interrupts execution until the graph is manually closed. Lastly, the display range is automatically adjusted to the range of values present in the image. A very simple example is:

```
plt.imshow(starry_img)
plt.show()
```

If we need to show a grey image (stored in a variable `grey_img`) using a grey colormap, then we have to write:

```
plt.imshow(grey_img, cmap='gray')
plt.show()
```

Save an image

Using OpenCV, you can save the image

```
cv.imwrite('new_starry_night.png',starry_img)
```

Write a code to save the image if the pressed key, i.e. `k`, corresponds to the code of "s", i.e. `ord('s')`.

3. Exploring the image

Load the image starry_night.jpg

The variable containing the image can be handled as a matrix. You can access image elements by indexing the matrix.

Access the pixel in [100,100] position and display it (*print*).

What do you get? What does it mean? How does python code color images? What are the R, G, and B values of the pixel?

Copy the image in a new variable and modify the value of the [100,100] pixel to white and show it

Can you see the white pixel?

Access image properties

Once the image is contained in a variable you can access to several properties:

```
starry_img.dtype
starry_img.size
starry_img.shape
```

Have a look to these properties. Do you understand all the values retrieved?

You can print the property directly or save them in a variable

```
rows, cols, chans = starry_img.shape
```

Editing and extracting ROIs (Region Of Interest)

Assuming that the image is a matrix you can extract a region by indexing ranges of rows and columns. Extract region [30:190,600:730] and save it in a new variable named as *moon* and visualize it. Do you understand what the range limits mean?

Copy *starry_img* in a new variable and copy the extracted region moon in the [120:280,300:430] range of the new image. Visualize it.

How many moons does the image contain?



Two (many) moons

TIP	*Use* `cv.destroyAllWindows()` *to close all the opened windows anytime you need.*

Resize images

When an image is resized, there is image information that is lost (if the output size is smaller) or newly created (if the output size is larger). Interpolation options permit to control the way the pixel values are modified in these situations. OpenCV permits to perform resizing by `cv.resize`. Have a look to the help information of this command.

Resize *starry_img* to half size using `INTER_CUBIC` interpolation. Remember that this constant value is only define into the namespace of the OpenCV library so you have to refer it as `cv.INTER_CUBIC`. Visualize it.

4. Playing with colors

Load the image *primary_colors.png*.

In order to extract the color planes B, G, R the matrix is to be splitted:

```
b,g,r = cv.split(primary_colors_img)
```

Show both, original image and the images corresponding to the three primary colors. Do you understand the information you get?

Contrarily, you can compose a color image combining three-color planes properly.

```
cv.merge((b,g,r))
```

Modify color planes

Make null the blue plane of the image and show the new color image.

```
b.fill(0)
```

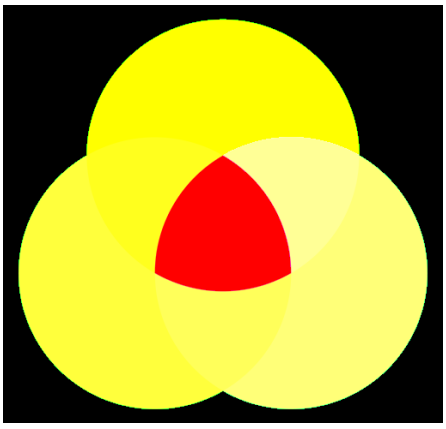
TIP	*Take into account that* `b.fill(0)` *changes* `b`*, i.e. you don't need to save the result of* `b.fill(0)` *in a new variable since the actual value of* `b` *is modified.* `fill` *is a method of the object.*

Change the image type

OpenCV permits to change the type of the image. More specifically, `cv.cvtColor` is to be used. Have a look to the help of the command and:

Convert the image to gray level. Visualize it.

Convert the image to HSV. Visualize it. What do you get?



HSV image shown as it was RGB coded

Extract H, S, V planes and visualize them. What do you get?

5. Adding and blending images

Libraries: *numpy*

```
import cv2 as cv
import numpy as np
import sys
import os
```

When working with OpenCV and pixel values (gray or color) attention should be paid to the arithmetic operations between scalars and the limitation of the scale range [0,255].

Have a **quick** look to the description of the *numpy* library to get an idea about its potential contents.

Execute this piece of code

```
x = np.uint8([250])
y = np.uint8([10])
print (cv.add(x,y))
print (x+y)
print (250+10)
```

Do you understand the values you get? Try to explain the differences between the three options.

Open *ml.png* and *upna_logo.png* images

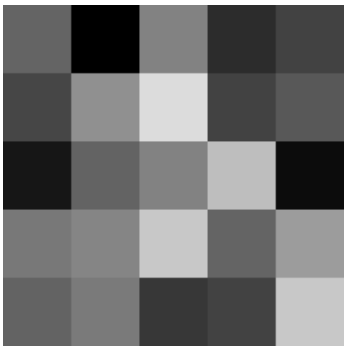
Resize *upna_logo* image using scale factors (fx=0.11 and fy=0.11) and `INTER_CUBIC` interpolation. What is the shape of the resized image?

`INTER_CUBIC` is a type of interpolation. When an image is resized the amount of information in the image varies, e.g. if the size is increased "new" pixel values are obtained from the intensity values of the surrounding pixels. For more information see this link

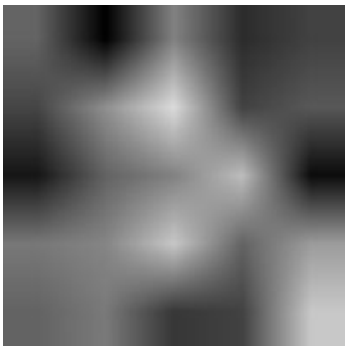
Copy the following piece of code:

```
orig_image = np.array([[100,0,130,44,66], [70,144,220,66,88],
[22,99,130,190,12],[120,133,200,100,156],[99,122,55,66,200]],dtype=np.
uint8)
```

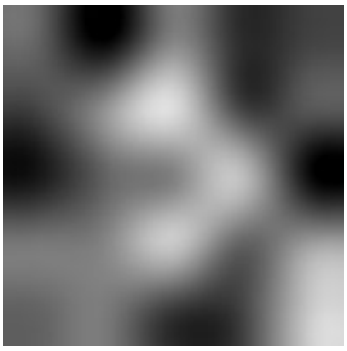
Resize the image to size (200, 200) using `INTER_NEAREST` `INTER_LINEAR` and , `INTER_CUBIC` interpolation. Visualize the results. Do you think `INTER_NEAREST` interpolation result correct? It seems to be visually the same as the original image. Why?



`INTER_NEAREST`



`INTER_LINEAR`



`INTER_CUBIC`

Zero padding for images

In order to add (sum up) two images their sizes should be the same (matrices are summed up).

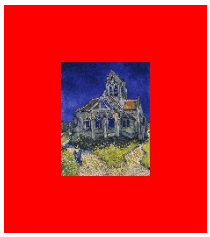
Since the new *upna_logo* is smaller we need to add null columns and rows in order to combine it with *ml.png* image, i.e. a zero padding needs to be performed (`cv.copyMakeBorder`). Have a look to the help of the command to understand the input parameters.

```
pad_logo_img = cv.copyMakeBorder(small_logo_img,
      ml_rows-small_logo_rows,0,ml_cols-small_logo_cols,0,
      cv.BORDER_CONSTANT,value=0)
```

In the example below the image is resized by including 0s (`value=0`) but this is not the only option. The command `cv.copyMakeBorder` provides alternatives. How would you achieve to fill the image using red color?

If you explore the options of `cv.copyMakeBorder` you will find other option such as `cv.BORDER_REFLECT`, `cv.BORDER_WRAP`, and `cv.BORDER_REPLICATE`.

Open image *church_small.jpg* and increase its size by adding 400 columns in left and right sides and 400 rows in the upper and lower limits. Try the alternative parameter values.



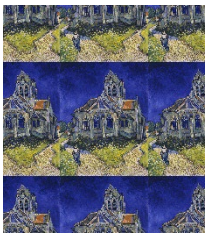
`cv.BORDER_CONSTANT`



`cv.BORDER_REFLECT`



`cv.BORDER_REPLICATE`



`cv.BORDER_WRAP`

Adding images

`cv.Add` permits to sum up images. Sum up *ml* and padded *upna_logo* images and visualize the result.

TIP	*Have a look to the help information of the commands.*

Blending images

`cv.AddWeighted` permits to sum up images weighting differently the inputs. Sum up *ml* and padded *upna_logo* images, given a *weight*=0.5 to both images. Visualize the result.

TIP	*Have a look to the help information of the commands.*



Blended image

Copy ROI

Finally copy the resized *upna_logo* image as a ROI in *ml_image*.

TIP	*See section 3: edit and extracting ROIs.*

What is the difference between the three operations?

6. Medical Images

Libraries: *pydicom* , *pynrrd*, *scipy*

```
import cv2 as cv
import pydicom as dicom
import numpy as np
import sys
import os
from scipy.io import loadmat
```

Several medical image files have .dcm extension. The extension dcm comes from DICOM (Digital Imaging and Communication On Medicine) which is a standard for medical data. Special commands are required for handling DICOM-type images.

```
aux = dicom.dcmread('pathtodcmimage')
knee1_img = aux.pixel_array
```

Open *knee1.dcm* and *knee2.dcm* and visualize them. They are Magnetic Resonance (MR) images of a knee showing different information.

The range of medical images could be slightly larger than the one of a standard image. Before doing further processing the range needs to be normalized (have a look to the `cv.normalize` documentation in order to understand what you are doing):

```
knee1_img = cv.normalize(knee1_img, None, alpha=0,beta=1,
                         norm_type=cv.NORM_MINMAX, dtype=cv.CV_32F)
```

With the option `cv.NORM_MINMAX`, we are normalizing the input image to an output range between 0 (alpha value) and 1 (beta value), using a 32 bits data floating representation, i.e., the minimum value of the input image is associated with 0 in the output image and the maximum value of the input image is converted to 1 in the output image.

Another common file extension for medical image is .nrrd. We have installed in our environment a specific library to read them: *pynrrd*.

```
import nrrd
img,header = nrrd.read('pathtonrrdimage')
```

Variable `img` is a 3D array containing the gray level information of the image. If the original image is 2D, the third dimension of this array is 1. In order to remove these axes of length one you can utilize the *numpy* function `squeeze()`. Variable `header` contains extra information about image such as spatial origin and orientation, encoding, etc. We are not going to use this information in our lab sessions. It is also recommendable to normalize our nrrd images, as shown before.

Image fusion

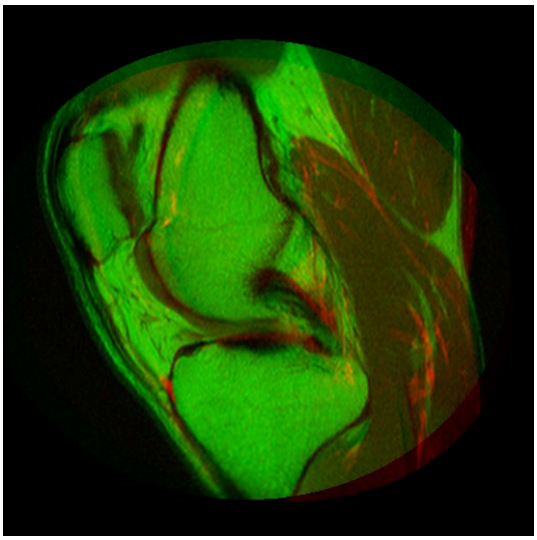
Apart from blending/adding images we can also "fuse" information coming from different images. This is a standard procedure in fields such as medical imaging in which data coming from different modalities can be combined by means of a fusion procedure.

One of the simplest fusion techniques is to combine images by inserting them in color planes of an image color. The individual images to be fused are inserted in RGB planes. Then the color image is a combination of these planes.

Combine knee1 and knee2 in a single color image. The fusion consists in assigning knee1 and knee2 to green and red channels respectively. The blue channel is a null plane of the same size:

```
np.zeros(g_knee1.shape, dtype=g_knee1.dtype)
```

TIP	*See section 4: playing with colors.*



A fusion of two knees

Reading 3D information

The use of 3D and 4D image information is a standard in the field of medical imaging. The loaded image is a volume instead of a single 2D plane. The volume is a combination of stacked slices (2D planes). In order to read a volume, a specific command is to be used.

```
aux = loadmat('pathtomri/mri_01.mat')
mri_vol = aux['mri_01']
```

Note that the volume you are using is a .mat file, i.e. Matlab file. So, you can play with both tools if needed.

Have a look to the shape of *mri_vol*? Do you understand these values? What is the type of information?

As in the previous case, the type of data cannot be straightforwardly shown due to the range of values of the format employed. A possible option is to normalize data before showing it, using this formula:

$$mri_vol_{norm} = \frac{mri_{vol} - \min(mri_vol)}{max(mri_vol) - min(mri_vol)}$$

Analyze the equation. What is the effect of applying this equation to the image?

Instead, you can normalize the volume using `cv.normalize()` as done just before.

Implement normalization in python for *mri_vol*.

In order to show a specific slice, the 3D matrix needs to be indexed

```
mri_slice = mri_vol_norm[:,:,6]
```

Now, you can visualize the slice.

Image registration

Load the volumes *mri_01.mat, ct_01.mat* and *pet_01.mat*. Normalize them and visualize any of the slices.

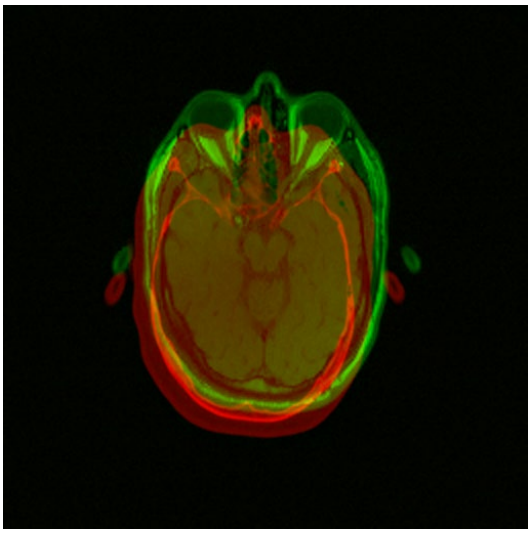
Fuse ct_01 and mr_01 using the same technique used before for knee images. In this case, CT (Computed Tomography) image is used as R while MR is used as G. Channel B is again 0.

Use the following slices of the volumes

```
mri_slice = mri_vol_norm[:,:,6]
ct_slice = ct_vol_norm[:,:,7]
pet_slice = pet_vol_norm[:,:,4]
```

In this case, you have to resize the MRI slice to the size of the CT slice. Use `INTER_CUBIC` interpolation shown before.

What is the result?



CT and MRI misaligned brains

The images are not aligned. As you can observe in this case the misalignment is evident. This is frequent in medical image fusion. Take into account that the patient cannot lay in exactly the same position during the image acquisition processes in both machines. Before the fusion can be carried out the images need to be registered.

Try the following piece of code and try to understand each line:

```
ct_rows, ct_cols = ct_slice.shape
x_slider_min = -100
x_slider_max = 100
y_slider_min = -100
y_slider_max = 100
theta_slider_min = -30
theta_slider_max = 30
window_name = 'Interactive window'
cv.namedWindow(window_name)
x_track_name = 'X_Shift'
y_track_name = 'Y_Shift'
theta_track_name = 'Theta_Rotation'
val_x = val_y = val_theta = 0

def onchange_x(val):
    global val_x
    val_x = val
    onchange_any()
def onchange_y(val):
    global val_y
    val_y = val
    onchange_any()
def onchange_theta(val):
    global val_theta
    val_theta = val
    onchange_any()
```

```
def onchange_any():
    global mri_slice_resized,ct_slice,ct_cols,ct_rows
    global x_track_name,y_track_name,theta_track_name
    global window_name
    global val_x,val_y,val_theta

    realval_x = x_slider_min+(x_slider_max-x_slider_min)*val_x/100
    realval_y = y_slider_min+(y_slider_max-y_slider_min)*val_y/100
    realval_theta = theta_slider_min + \
        (theta_slider_max-theta_slider_min)*val_theta/100
    realval_theta = np.pi*realval_theta/180
    M = np.float32([[np.cos(realval_theta),np.sin(realval_theta),
                realval_y], [-np.sin(realval_theta),
                np.cos(realval_theta),realval_x]])
    dst = cv.warpAffine(mri_slice_resized,M,(ct_cols,ct_rows))
    g_fus = dst
    r_fus = ct_slice
    b_zero = np.zeros(ct_slice.shape,dtype=ct_slice.dtype)
    fus_img = cv.merge((b_zero,g_fus,r_fus))
    cv.imshow(window_name,fus_img)

cv.createTrackbar(x_track_name,window_name,50,100, onchange_x)
cv.createTrackbar(y_track_name,window_name,50,100, onchange_y)
cv.createTrackbar(theta_track_name,window_name,50,100, onchange_theta)

cv.waitKey()
cv.destroyAllWindows()
```

LAB SESSION 2
IMAGE ENHANCEMENT AND FILTERING

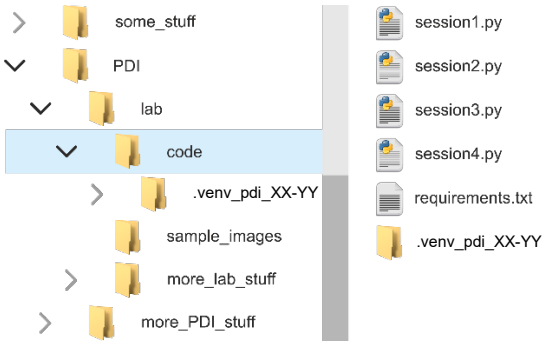
1. Preliminaries

We recommend you employ a single file for the whole session. In this manner, you can import the required libraries in the heading of the file and you don't need to load the libraries for every single file created during the lab session. Remember that you can comment sections of code to focus in the alternative exercises provided.

Open VSC and activate the *.venv_pdi_24-25* environment. It is assumed that you have done session 0 and 1 successfully and that you are familiar with the setting up procedure of the environment. If not, go back to session 0.

TIP	*Remember that all the images required for the session are contained in sample_images folder that you need to download in your computer.*
TIP	*For those of you working with lab computers, remind to use the environment contained in the USB key.*

So, the final folder tree in your computer could look like this:



Please, double check that the structure of the folder in your computer follow the above scheme.

TIP	*It is convenient to change the working path to your current working directory. In this manner, relative paths can be used and the code can be more easily exported to other machines.*

```
import os
#Getting the path for this file
project_path = os.path.dirname(__file__)
#Changing out current working directory. In this way, we can work with
#relative paths
os.chdir(project_path)
```

Note that `__file__` is a predefined variable returning the path of our script. The variables or methods defined with **two** underscores on each end are usually named as a *dunder*. You can consult this link to get a detailed explanation about using underscores in Python.

TIP	*You can comment lines of code using #. When you need to comment several lines of code use ''' at the beginning and end of the commented code.*

2. Calculate image histogram

Libraries: *cv2, numpy*, *matplotlib*

```
import numpy as np
import os
from matplotlib import pyplot as plt
import cv2 as cv
```

Open image *mri_head_sag.jpg* and visualize it.

```
head_img = cv.imread(pathtotheimage)
```

Is it a color image? How can you be sure?

Convert it to gray levels

TIP	*Revisit session 1*

Calculate the histogram:

In order to calculate image histogram alternative options can be used.

a) *pyplot* provides the option to calculate image histogram, to this end the image must be converted into a vector.

```
head_img_bw.ravel()
```

Check help of *pyplot* to calculate histogram, the number of bins is 256 and the range is [0,255]

b) Another way to calculate image histogram is to use OpenCV.

Check help of OpenCV to calculate the histogram and visualize it. Use, when calling function `calcHist()`, the parameter `range=[0,256]`. Here you can check the full explanation.

The difference with respect the previous point is that the result of the OpenCV function is not a graph but a vector that should be further plotted in order to be visualized. You can use the function `plot()` of the *pyplot* library to show this histogram vector:

```
plt.plot(hist)
plt.show()
```

c) *numpy* also helps to calculate the histogram, to this end the image has to be converted into a vector

```
head_img_bw.flatten()
```

Check help of `histogram()` function to calculate histogram; the number of bins is 256 and now the range is [0,255].

Apparently flatten and ravel perform the same operation on your data, however, in this link the differences between these two commands are detailed.

Cumulative histogram

In order to calculate the cumulative histogram further operations are needed. Assuming that your histogram is saved in *hist* variable, first the Cumulative Distribution Function (CDF) is needed.

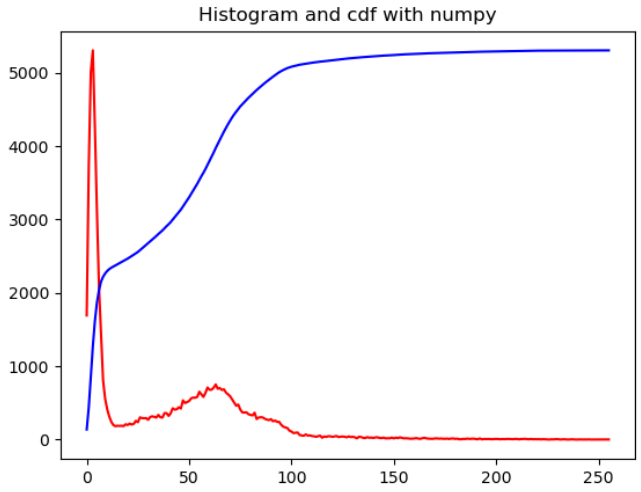
```
cdf = hist.cumsum()
```

Before plotting the cumulative histogram it is usual normalizing the scale by:

```
cdf_scaled = cdf*hist.max()/cdf.max()
```

Execute this piece of code

```
plt.plot(hist, color='r')
plt.plot(cdf_scaled, color='b')
plt.title('Histogram and cdf with numpy')
plt.show()
```

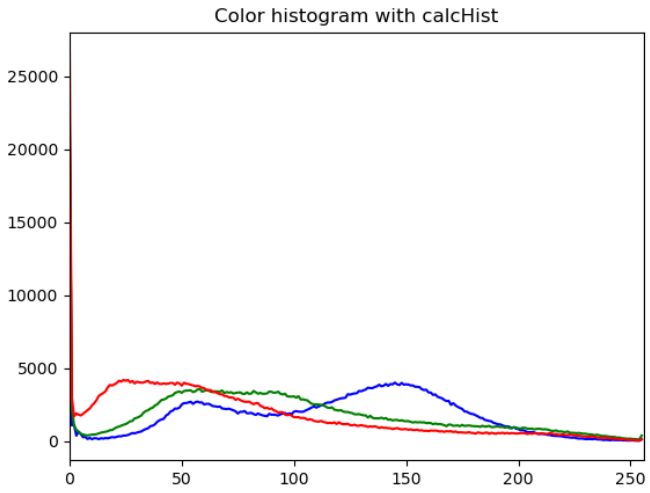


Histogram and cumulative histogram using numpy

What do you get? Notice that `plt.show()` displays all the contents assigned to *plt* object.

Open the image *starry_night.jpg.* Our objective now is to calculate histogram of a color image. Read again the online documentation to calculate the histogram and try to find out how the alternative color channels histograms can be obtained.

By means of a *for* loop, plot the histogram of blue, green and red channels.



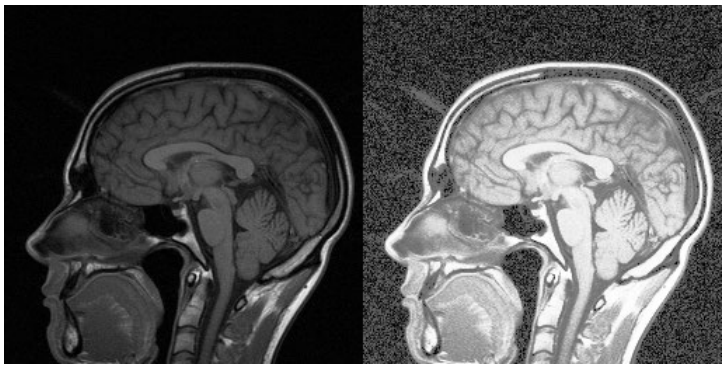
Histogram of a color image

Equalize image:

Read the help of `equalizeHist()` function from OpenCV. Once you understand the idea behind, equalize the gray level version of *mri_head_sag.jpg* image.

If you'd like to visualize images side by side in the same window you can use `np.hstack((image1,image2))` to arrange the images horizontally.

Now, display the images vertically arranged.



MRI image with its equalization

Local equalization:

Open *hidden-symbols.tif* and equalize it.

Have a look to `createCLAHE()` command of OpenCV. The parameters of `createCLAHE()` command are `tileGridSize` and `clipLimit`. As you know, local histogram equalization may cause noise to be amplified in near-constant regions. Contrast Limited AHE (CLAHE) is a variant of adaptive histogram equalization in which the contrast amplification is limited, so as to reduce this problem of noise amplification. `clipLimit` permits to establish a

limit to clip the CDF. The algorithm does not dismiss the data over the limit but redistribute it in other bins. With `createCLAHE()` you construct the filter object. Do not forget to use its method `apply()` with the input image to obtain the output one.



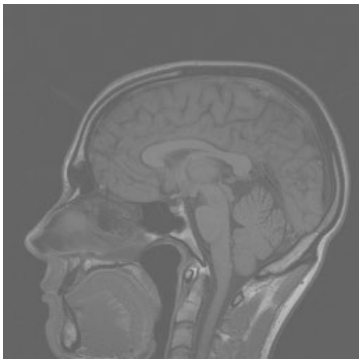
Example of image enhanced with CLAHE

Compare global and local equalization for the image *hidden-symbols.tif* and *mri_head_sag.jpg*.

3. Contrast stretching

Open image *mri_head_sag.jpg* and convert it to gray level. Execute the following piece of code:

```
original = head_img.copy()
xp = [original.min(), original.max()]
fp = [100,200]
x = np.arange(256)
table = np.interp(x, xp, fp).astype('uint8')
wash_head = cv.LUT(head_img,table)
cv.imshow("original", original)
cv.imshow("Washed head", wash_head)
cv.waitKey(0)
cv.destroyAllWindows()
```



"Washed" head

As you can see the new image *wash_head* presents lower contrast with respect to the *original* one. Our objective now is to consider *wash_head* as the image to be enhanced. More specifically, a contrast stretching will be performed.

The proposal is to stretch the range of the image, i.e.to expand minimum and maximum gray levels of the image, to [0,255] interval.

The following OpenCV instruction achieves to stretch the image to the pursued range.

```
#Using normalize
norm_head = cv.normalize(wash_head,None,alpha=0,beta=1,
                         norm_type=cv.NORM_MINMAX,dtype=cv.CV_32F)
#scale to uint8
norm_head = (255*norm_head).astype(np.uint8)
```

In the first part of the code the range of the image is scaled to the range [0,1]. The second instruction scales the dynamic gray level range to [0,255] and converts it to uint8.

Have a look to the help of `normalize()`. Perform the same contrast stretching using `normalize` exclusively.

Compare also the previous results with the output of the equalization of *wash_head* image.

Visualize in the same window the *original* image, *washed_head* and *norm_head* images.

4. Spatial convolution

The OpenCV generic instruction to perform spatial filtering is *filter2D.* Have a look to the documentation.

Average filter

Perform a spatial filtering of *starry_night.jpg* (converted into gray level) using an averaging filter of size 5x5.

Visualize original and filtered images.

Increase and decrease the size of the filter to observe the effect in the image.

 TIP	*It is convenient to create the filter previously; np.ones is an easy way to create simple matrices.*

Gaussian blurring

Additionally, OpenCV provides predefined filters such as `GaussianBlur()`. Smooth *starry_night* image with a varying filter size.

Laplacian

Laplacian filtering can be also performed using *Laplacian*. It is known that Laplacian operator has a poor behavior with noisy images. So it is always mandatory to filter low-pass the input image and then apply the Laplacian algorithm.

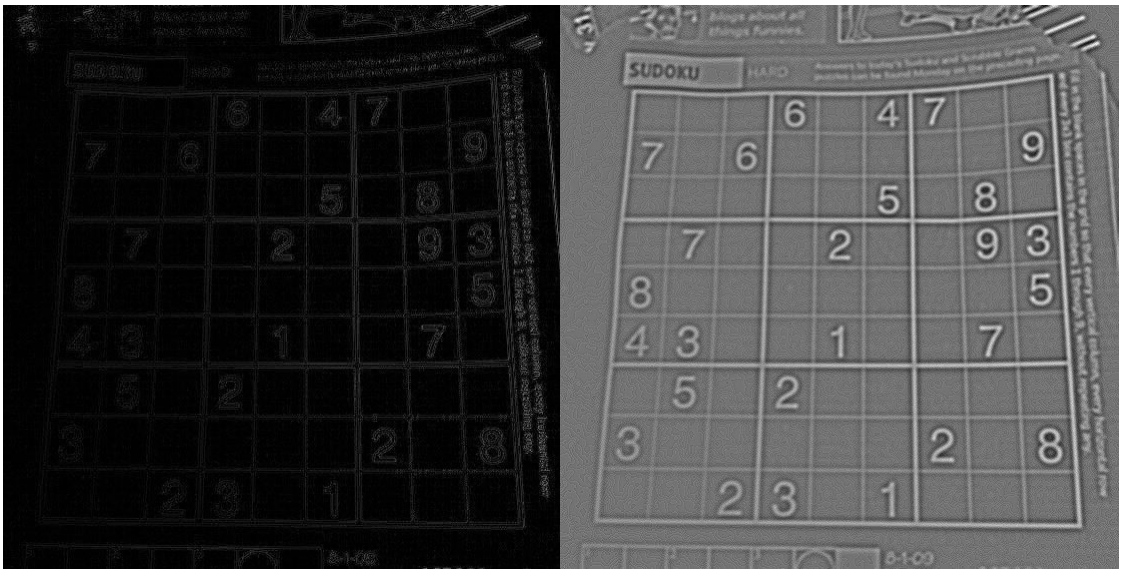
Using *sudoku*.png file as input image, apply first a *Gaussian* blurring filter and then, with this intermediate result as input image, calculate the *Laplacian* final output.

Does this combination of algorithms, i.e. *Gaussian* blurring and *Laplacian*, sound familiar? What is the name of the algorithm?

One of the problems of Laplacian is that it can result in negative values that `cv.imshow()` cannot display correctly. We have two approaches to manage this issue:

1. Use `convertScaleAbs()` operation from OpenCV to bring it to positive values.
2. Use `normalize()` function, as we have learned in our first lab session.

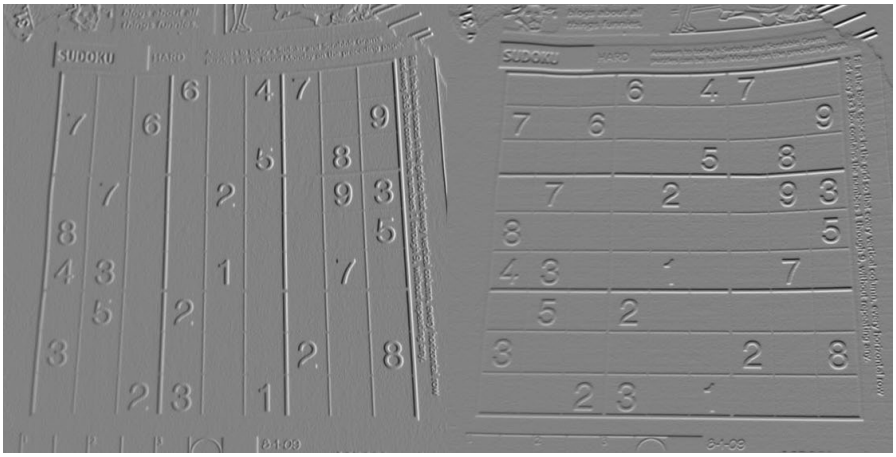
Visualize the Laplacian using these two approaches and compare them.

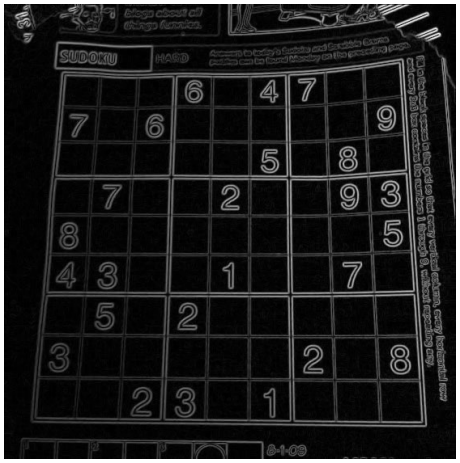


Laplacian images using `convertScaleAbs()` and `normalize()`

Sobel

Find out the way in which OpenCV calculates the Sobel filtering of the image and apply it to *sudoku.png.*





Sobel filtering: (x, y) components and magnitude

Now, perform a Gaussian blurring to the image before calculating Sobel. What do you get? What do you think could be the objective of performing a smoothing before edge detection?

Median

Open the image *circuit.jpg* and apply Gaussian blurring. The noise affecting the image is bipolar/salt&pepper noise. As you know median filtering is the most appropriate in this case. Find out how OpenCV performs median filtering and apply it. Compare the results between Gaussian and median filtering.

5. Discrete Fourier Transform (DFT)

OpenCV provides the functions `cv.dft()` and `cv.idft()` to calculate the 2D Fourier direct and inverse transforms.

Check carefully the flags of the command and take into account that a complex matrix is expected as output.

According to OpenCV, the image needs to be converted to float32, i.e. `np.float32(image)`

Have a look to the help of the command.

Remember that it is convenient to modify the image spectrum in order to visualize a centered version of the transform. *numpy* provides `fft.fftshift()` to this end.

```
np.fft.fftshift(spectrum)
```

How does `fft.fftshift()` center the spectrum? What do you think?

Let's see it more carefully, create the following two images.

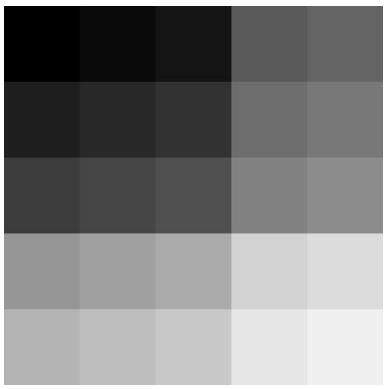
```
orig_spect_odd= np.array ([[0,10,20,90,100], [30,40,50,110,120],
[60,70,80,130,140], [150,160,170,210,220], [180,190,200,230,240]],
dtype=np.uint8)

orig_spect_even=np.array([[0,7,14,63,70,77], [21,28,35,84,91,98],
 [42,49,56,105,112,119], [126,133,140,189,196,203],
 [147,154,161,210,217,224],[168,175,182,231,238,245]],dtype=np.uint8)
```

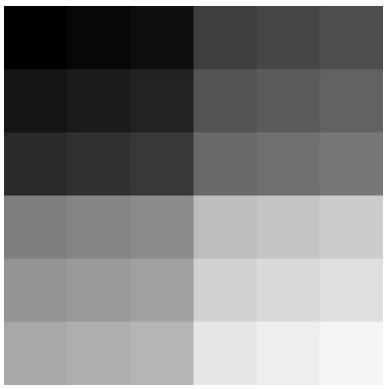
What is the size of these spectrums?

Apply `fftshift` and visualize the results. The size of the spectrums is too small. You can always increase the size of the visualization window by using `cv.resizeWindow.`

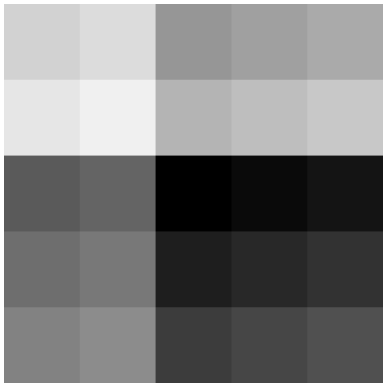
	*On Mac this function does not work properly. To get the above figures, resize first the image to their final size, using `INTER_NEAREST` interpolator, and then visualize them normally with `imshow()`*



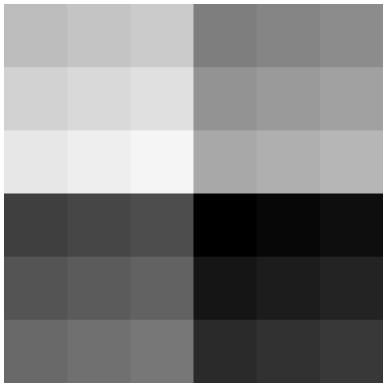
Original odd



Original even



Original odd shifted



Original even shifted

Command `dft()` retrieves 3D matrix containing the real and the imaginary parts of the spectrum. The module and phase of the spectrum can be calculated using `cartToPolar`.

```
mag, phase = cv.cartToPolar(real_part, imaginary_part)
```

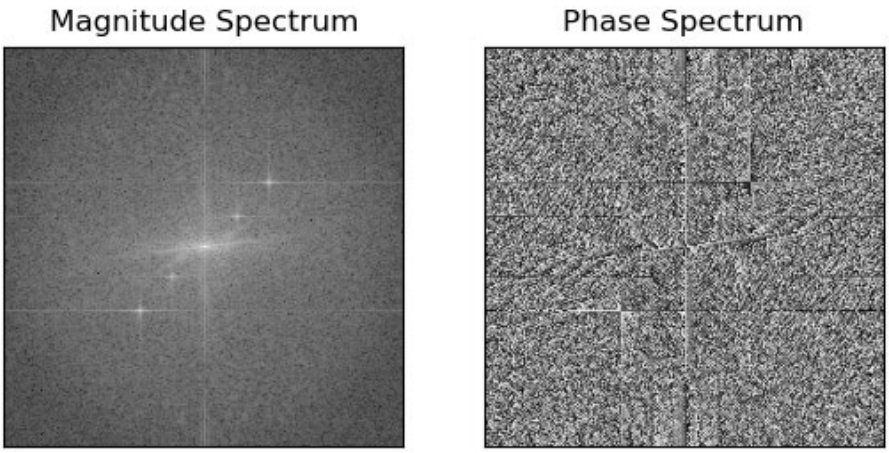
Calculate image transform:

Open *noisyknee.png* image to calculate its transform. Extract both parts from the spectrum and visualize them.

The range of the module contains too large values that disrupt the visualization of the spectrum. A logarithmic transform can be applied in order to improve it.

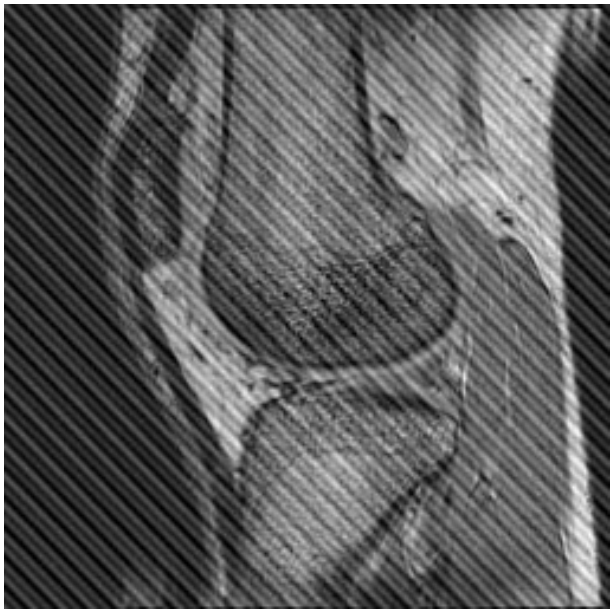
```
mag_db=20*np.log(mag)
```

Visualize the modified module of the spectrum. In order to show properly the magnitude and the phase of the DFT, you can use the function `imshow()` from the *matplotlib* library. Remember adding the parameter `cmap='gray'` into this function.



Magnitude and phase spectrum

Can you extract any significant information from the magnitude that is confirmed in the original image?



noisyknee.png

What is the effect of removing `cmap='gray'` option?

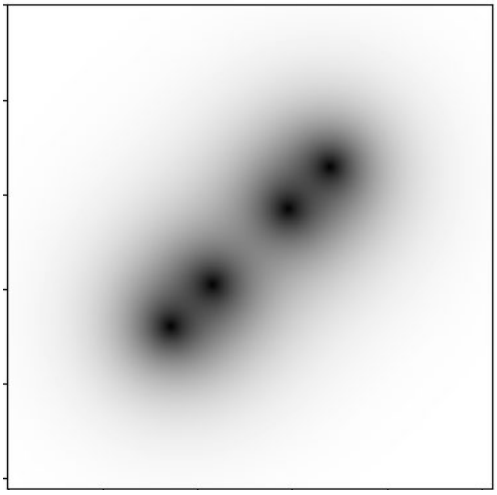
Create a function named as `visualizespectrum()` that has as input an image. The function should retrieve the image spectrum and show it.

In your *sample_images* folder you will find files named as *carpet_**.jpg*. They are samples of alternative textures. Calculate their spectra and try to find a connection between the image and the obtained magnitude of the spectrum.

Filtering in frequency domain

In auxiliaryfiles (miAulario) you will find a .py file containing a piece of code to implement a reject notch filter. Include it in your code and select the right parameters (filter size, centers and cut-off frequencies for each notch).

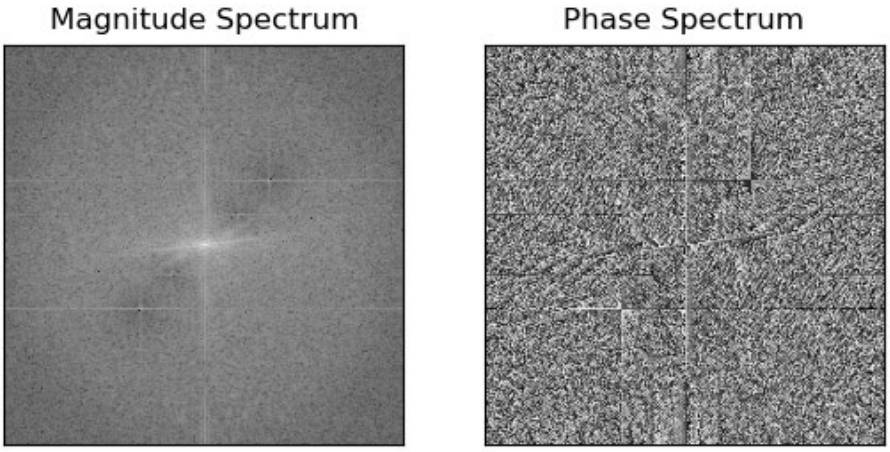
Visualize the result.



Notch reject filter

Now, read the code again and try to understand each command.

The mask created can be used as a filter in frequency domain. Apply it to our image spectrum, working with its real and imaginary parts. Have a look to the spectrum after filtering.



Knee spectrum after filtering

The centering procedure performed for the spectrum should be reversed before moving to the spatial domain.

```
f_ishift = np.fft.ifftshift(spect_filtered)
```

Calculate now the filtered image in the spatial domain by using the inverse transform.

The spatial version of the filtered image contains complex numbers. It is recommended to calculate the module of the image. Use `cv.magnitude`.

Visualize the results. What do you get? How do you interpret the result?



Knee image after filtering

LAB SESSION 3
MORPHOLOGY

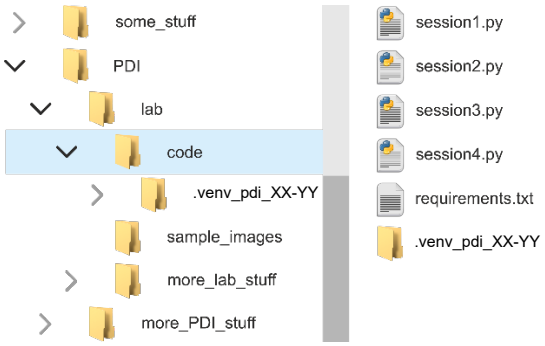
1. Preliminaries

We recommend you employ a single file for the whole session. In this manner, you can import the required libraries in the heading of the file and you don't need to load the libraries for every single file created during the lab session. Remember that you can comment sections of code to focus in the alternative exercises provided.

Open VSC and activate the *.venv_pdi_24-25* environment. It is assumed that you have done session 0, 1 and 2 successfully and that you are familiar with the setting up procedure of the environment. If not, go back to session 0.

TIP	*Remember that all the images required for the session are contained in sample_images folder that you need to download in your computer.*
TIP	*For those of you working with lab computers, remind to use the environment contained in the USB key.*

So, the final folder tree in your computer could look like this:



Please, double check that the structure of the folder in your computer follow the above scheme.

TIP	*It is convenient to change the working path to your current working directory. In this manner, relative paths can be used and the code can be more easily exported to other machines.*

```
import os
#Getting the path for this file
project_path = os.path.dirname(__file__)
#Changing out current working directory. In this way, we can work with
#relative paths
os.chdir(project_path)
```

Note that `__file__` is a predefined variable returning the path of our script. The variables or methods defined with **two** underscores on each end are usually named as a *dunder*. You can consult this link to get a detailed explanation about using underscores in Python.

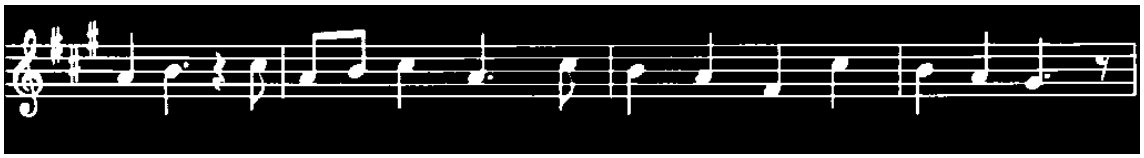
TIP	*You can comment lines of code using #. When you need to comment several lines of code use ''' at the beginning and end of the commented code.*

2. Basic operations: erosion, dilation, open and close

Libraries: cv2, numpy, os

```
import cv2 as cv
import numpy as np
import os
```

Open the image *notes_bin.png*.



Original image

Have a look to the help command of OpenCV to perform erosions, i.e. `cv.erode()`. Some of the inputs of the command are, as expected, the image to be eroded and the Structuring Element (SE). The SE is a binary matrix with 1 and 0 values.

Perform the erosion of the image using a square structuring element of size (5,5).



Erosion

Perform the dilation using the same element.



Dilation

Visualize the results.

In order to understand the effect of the structuring element size, modify it and check the results (try (3,3), (7,7), (9,9)…)

TIP	*It is convenient that SE elements are uint8 type*

The structuring element can be also constructed using a higher level command such as `cv.getStructuringElement()`. The input elements are the shape (`MorphShapes`) and its size. Now, construct the square SE using this alternative and compare the resulting erosion and dilation with the previous results.

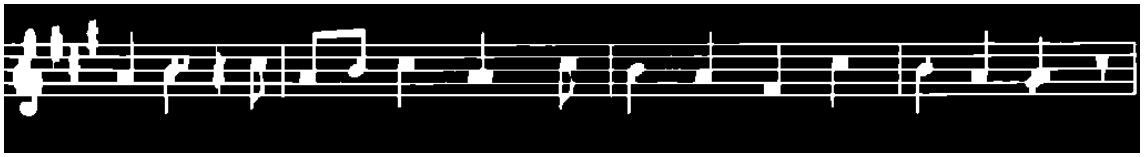
TIP	*`ksize` in `cv.getStructuringElement()` has format (cols, rows)*

The comparison can be carried out numerically, e.g. compute the difference or visually (arrange vertically the images for the comparison in the same window).

As you already know, open and close operations are concatenations of erosion+dilation (open) and dilation+erosion (close) operations. The function `cv.morphologyEx()` is a general-purpose instruction to execute morphology commands, as one of the inputs is the morphological operation to perform. Have a look to the help.



Open

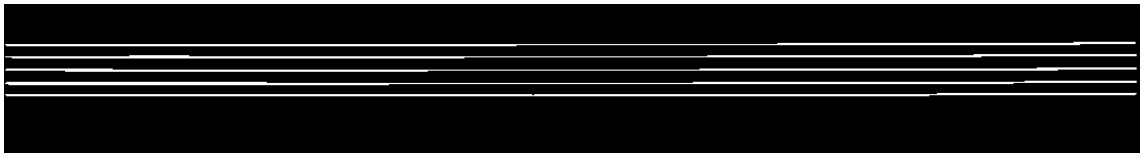


Close

Using the same SE perform the open and close operations on the same image. What is the difference with respect to erosion and dilation?

In the following lines some explanatory operations are shown.

Construct a rectangular SE of size (51,1) and perform an open operation on the notes image. What do you get? The result is related to the shape of SE? Why? Try to explain the results.



Open with SE (51,1)

Now, try to obtain as result the bar lines of the notes using opening.



Bar lines

Finally, detect note heads.



Note heads

3. Advanced operations

Image reconstruction

Image reconstruction is one of the applications of geodesic morphological reconstruction. The objective of this exercise is to extract from the image those objects having vertical lines, i.e, notes, bar lines….

In the following lines, reconstruction pseudocode is given. Translate it into python:

TIP	*In order to do the following exercise a good understanding of geodesic reconstruction is definitely required. This is a strong recommendation…*

MAIN

```
#construct structuring element
se_1_5 = StructuringElement(rect, (1,5))
#open_reconstruction (image, SE, n) is a python function that returns
reconstruction by opening. You need to create this function in python
where n is the number of erosion needed to create marker image
OR_test = open_reconstruction(notes_bin, se_1_5, 1)
show(notes_bin)
show(OR_test)
end
```

OPEN_RECONSTRUCTION

```
#open_reconstruction function pseudocode
function result = open_reconstruction(image, se, n)
#construct marker using erosion
marker = erosion(image, se, n)
#basic_reconstruction(marker,mask,se, method) is a python function that
returns the image reconstruction (by erosion or dilation) provided the
marker, the original mask (image), se and an option method with possible
```

```
values: 'erode' or 'dilation' to select if the reconstruction should be
calculated by erosion or dilation. You need to create this function.
Result = basic_reconstruction(marker, image, se, 'dilate')
```

BASIC_RECONSTRUCTION

```
#basic_reconstruction function pseudocode
function result = basic_reconstruction(marker, mask, se, method)
#save marker in an auxiliary variable
prev_marker = marker
#calculate new_marker using a geodesic one step dilation/erosion
depending on method value. geo_step (marker, mask, se, method) is a
python function performing one-size geodesic dilation or erosion
depending on method. You need to create this function.
new_marker = geo_step(prev_marker, mask_se, method)
#while the result of the geodesic step is different to the one in the
previous iteration continue doing geodesic iterations (geo_step).
while prev_marker is different to new_marker
      prev_marker = new_marker
      new_marker = geo_step(prev_marker, mask, se, method)
end while
result=new_marker
```

GEO_STEP

```
#geo_step function pseudocode
function result = geo_step (marker, mask, se, method)
#if method is dilation or erosion perform the corresponding geodesic
operation
if method is 'dilate'
      result = and(dilation (marker, se), mask)
elseif method is 'erode'
      result = or(erosion(maker, se), mask)
else
      do nothing
end
```

TIP	*To calculate the logical operator and/or* `and/or` *logical operator or OpenCV* `cv.bitwise_and()` *and* `cv.bitwise_or()` *are provided*

The image to be obtained looks as follows:



OR_test

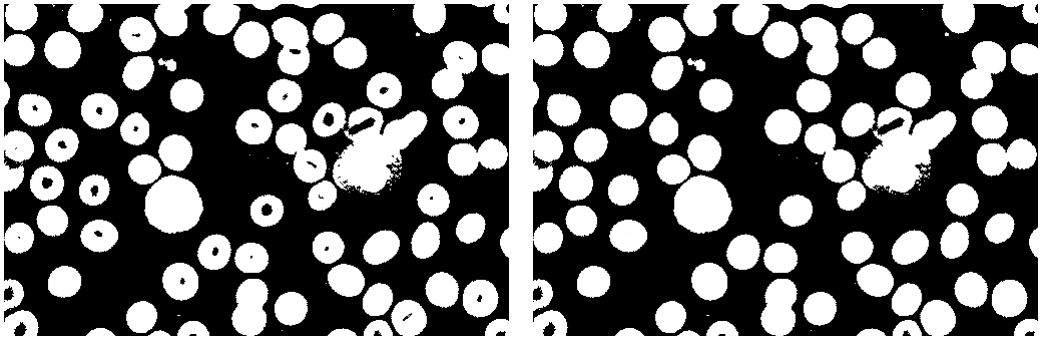
Optimize your code so that intermediate results are obtained. In order words, the results of the iterations carried out during the reconstruction process should be visualized. What would you change in the code to implement closing by reconstruction?

Change the size and shape of the structuring element when possible. Identify the key points in which SE is critical. Change its shape/size and check your hypothesis.

Hole filling

Using part of the code you have written in the previous exercise, implement a hole filling code for *malaria3_bin.png* image. We recommend implementing a function named `imfill(image)` having the image to be filled as the only input.

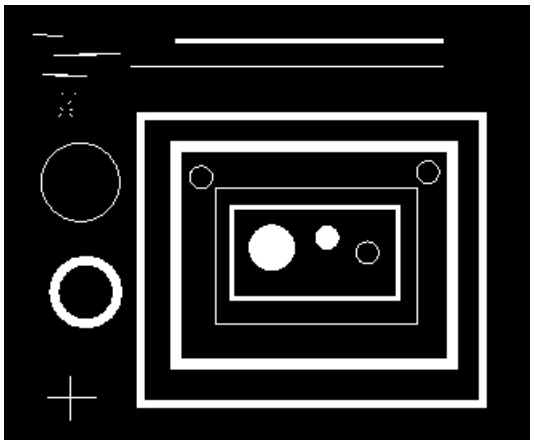
TIP	*In order to do the following exercise a good understanding of geodesic reconstruction and hole filling algorithm is definitely required. This is a strong recommendation…*
TIP	`aux_f = np.zeros(image.shape, dtype=np.uint8)` `aux_f[0,:] = aux_f[-1,:] = aux_f[:,0] = aux_f[:,-1] = 255`



Binary input image and its version with holes filled

Border clearing

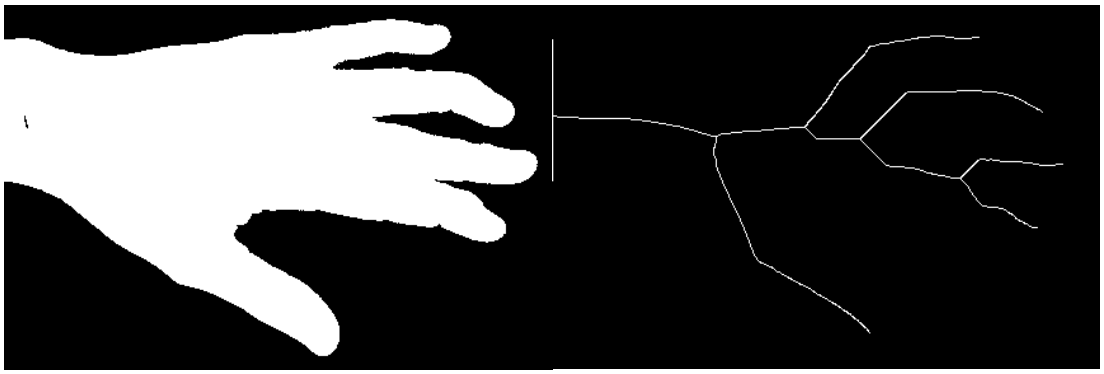
In the same manner implement `imclearborder(image)` function and show its performance with *blobs.png* image.



Border clearing

Skeleton/Thinning

Open *xhand_bin.png* image and convert it into grey level. OpenCV contains advanced/extended image processing techniques in the `cv.ximgproc` module. Have a look to the command and find the way to implement thinning.

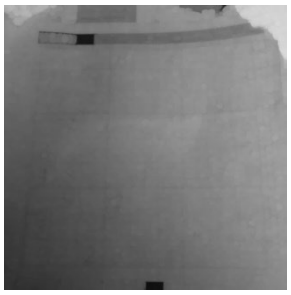


Original image and the thinning result

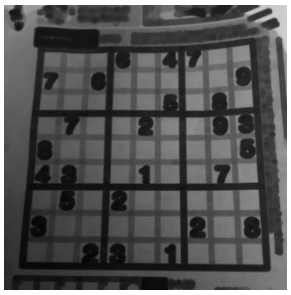
4. Extension to gray level

Until now BW images have been used as test images. As you know, morphology operations can be applied to gray level images. In this case, apart from the shape of the object the intensity of the pixels plays a role.

Open the image *sudoku.png* and apply dilation, erosion, open and close operations using a circular structuring element of size (11,11). Try to understand the result.



Gray level dilation



Gray level erosion



Gray level opening

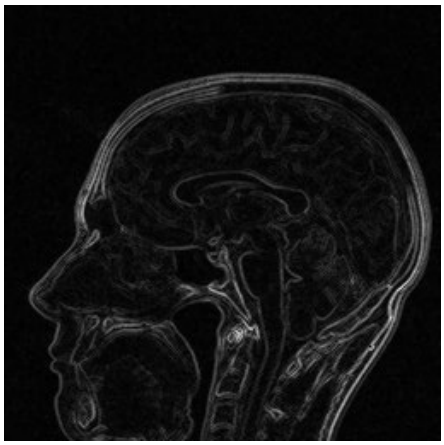


Gray level closing

The result of the dilation is a lighter image. Why the result of the erosion is darker?

Gradient

Using `cv.morphologyEx()` calculate the gradient of mri_head_sag.jpg image (remember you need to convert the image into gray level).



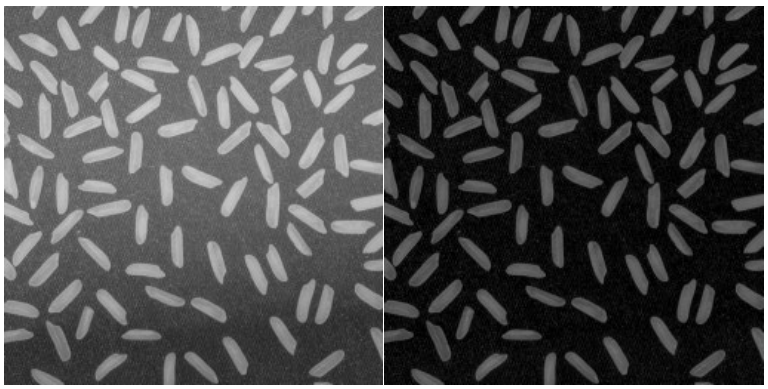
Gradient

Using basic morphological operators think and implement an alternative to obtain the image gradient.

Top-Hat

Top-hat is a useful morphological algorithm helpful to compensate for non-uniform background lighting images.

Apply top-hat (`cv.morphologyEx`) to *rice.png*. Can you appreciate the improvement in the image in terms of contrast?



Original and top hat images

5. Extraction connected components

When having a BW image it is important to identify the existing objects to perform further analysis. The labelling of the existing objects is technically named as connected components extraction.

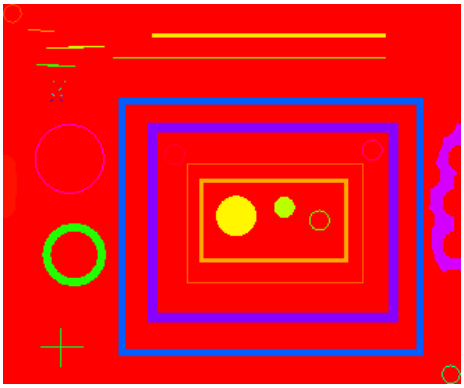
Given a BW image you can extract connected components using `cv.connectedComponents()`. Have a look to the command and extract connected components from image *blobs.png* (remember to convert it to gray level).

How many connected components does the image contain? (the result is 29)

You can use this function to visualize the labels and identify the corresponding components.

```
def ShowConnectedComponents(labels_im):
   h = np.uint8(255*labels_im/(1+np.max(labels_im)))
   s = 255*np.ones(labels_im.shape,dtype=h.dtype)
   v = s
   labeled_img_HSV = cv.merge((h, s, v))
   labeled_img_color = cv.cvtColor(labeled_img_HSV, cv.COLOR_HSV2BGR)

   cv.imshow('Labels color', labeled_img_color)
   cv.imshow('Labels', np.uint8(labels_im))
   cv.waitKey()
   cv.destroyAllWindows()
```



Connected components visualization

6. Advanced exercise: Convex Hull

The implementation of convex hull algorithm is proposed as advanced exercise.

The following function is provided as help.

```
def ch_one_axis(img, se):
   prev_img = img.copy()
   unstableQ = True
   while unstableQ:
      new_img = cv.bitwise_or(prev_img, cv.morphologyEx(
                  prev_img, cv.MORPH_HITMISS, se,
                  borderType = cv.BORDER_CONSTANT, borderValue = 0))
      cv.namedWindow('newimg', cv.WINDOW_NORMAL)
      cv.resizeWindow('newimg', 600, 600)
      cv.imshow('newimg', new_img)
      cv.waitKey()
      cv.destroyAllWindows()
      unstableQ = np.any(cv.absdiff(prev_img, new_img))
      prev_img = new_img
   return new_img
```

Review the procedure and apply it to *syntheticimage.png* image.

LAB SESSION 4
SEGMENTATION

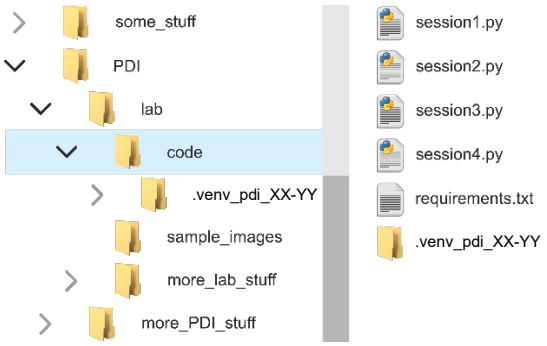
1. Preliminaries

We recommend you employ a single file for the whole session. In this manner, you can import the required libraries in the heading of the file and you don't need to load the libraries for every single file created during the lab session. Remember that you can comment sections of code to focus in the alternative exercises provided.

Open VSC and activate the *.venv_pdi_24-25* environment. It is assumed that you have done session 0, 1 and 2 successfully and that you are familiar with the setting up procedure of the environment. If not, go back to session 0.

TIP	*Remember that all the images required for the session are contained in sample_images folder that you need to download in your computer.*
TIP	*For those of you working with lab computers, remind to use the environment contained in the USB key.*

So, the final folder tree in your computer could look like this:



Please, double check that the structure of the folder in your computer follow the above scheme.

TIP	*It is convenient to change the working path to your current working directory. In this manner, relative paths can be used and the code can be more easily exported to other machines.*

```
import os
#Getting the path for this file
project_path = os.path.dirname(__file__)
#Changing out current working directory. In this way, we can work with
#relative paths
os.chdir(project_path)
```

Note that `__file__` is a predefined variable returning the path of our script. The variables or methods defined with **two** underscores on each end are usually named as a *dunder*. You can consult this link to get a detailed explanation about using underscores in Python.

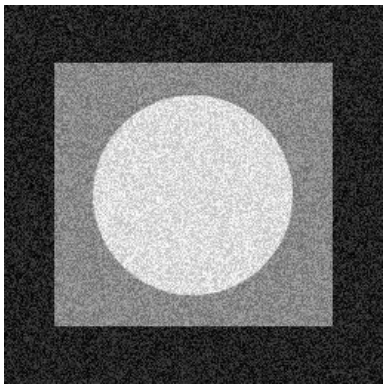
TIP	*You can comment lines of code using #. When you need to comment several lines of code use ''' at the beginning and end of the commented code.*

2. Basic thresholding

Libraries: cv2, numpy, os

```
import cv2 as cv
import numpy as np
import os
```

Open the image *geometry.jpg* and convert it to gray level.

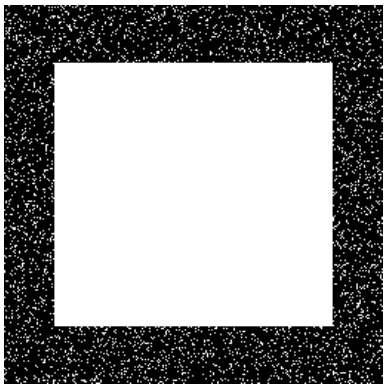


Original image

OpenCV provides tools to perform thresholding `cv.threshold()`. Have a look to the help and details of the command and try to explain the meaning of the input arguments. Pay special attention to the last argument describing alternative thresholding methods.

The objective is to differentiate the objects (square+circle) from the darker background. Segment the image using a threshold value of 60, i.e. pixel values above 60 should be converted into white.

What do you get?



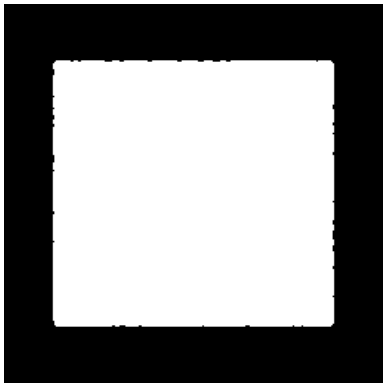
Segmentation TH=60

Now, perform an inverse segmentation, i.e. pixel values below 60 should be converted into black.

The result is not fully satisfying since the objects of interest are not properly isolated. The noise does not allow performing an acceptable segmentation.

One potential solution is to perform a low pass filtering before the thresholding. Apply a Gaussian blurring of size [9,9] and repeat the segmentation.

TIP	*Revisit lab session 2 for Gaussian blurring.*

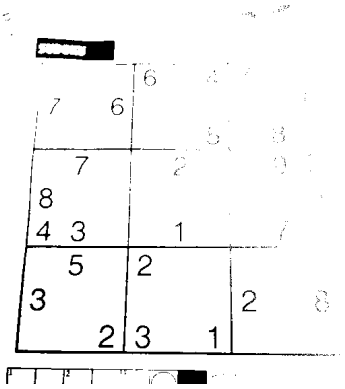


Segmentation (TH=60) after blurring

3. Adaptive Thresholding

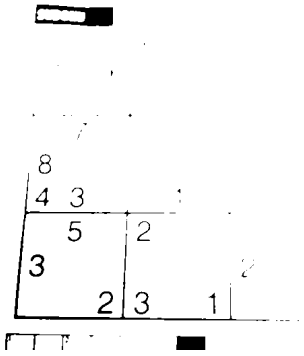
Now, open *sudoku.png* image and segment the image using a threshold value of 33. Our objective is to differentiate the numbers (and consequently square lines) from the background.

What do you get?



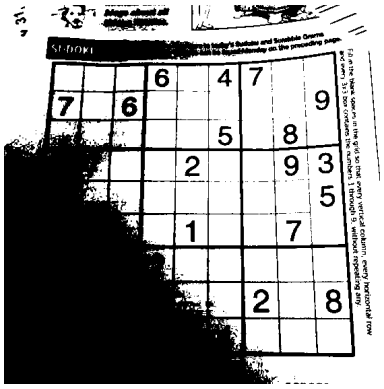
Segmentation (TH=33)

Apply Gaussian blurring to improve the result. Is it ok?



Segmentation (TH=33) after blurring

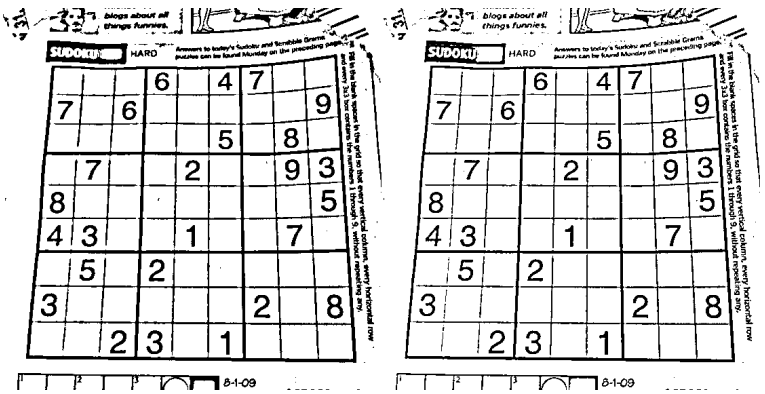
What if you use Otsu thresholding option?



Otsu result

Apparently, none of the methods provides satisfying results. All the methods suggested are global thresholding methods, i.e. they assume a single threshold value for all the pixels in the image. OpenCV provides adaptive thresholding tools: `cv.adaptiveThreshold()`. Have a look to this function and perform an adaptive thresholding to the image.

Perform an adaptive thresholding of the image using the alternative options of the function. You can compare a simple adaptive thresholding with a more elaborate version using Gaussian weighting.

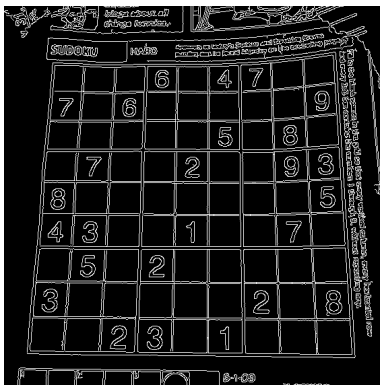


Result of adaptive thresholding (Mean vs. w.Gaussian)

4. Canny edge detection

Have a look to the command `cv.Canny()`.

Perform a simple Canny edge detection using as hysteresis thresholds with values TH_{low}=23 and TH_{high}=109 using *sudoku.png*. Observe that this function has a parameter (`apertureSize`) used to control the size of the mask that it is convolved with the input image to calculate the edge image. In the actual implementation of the OpenCV library, Sobel masks are used for this task, so the optimum mask obtained by Canny is not used in this version.



Canny

Try to find out what is the objective of the following piece of code:

```
val_thr_min = val_thr_max = val_asize = 0
def onchange_min(val):
      global val_thr_min
      val_thr_min = val
      onchange_any()
def onchange_max(val):
      global val_thr_max
      val_thr_max = val
      onchange_any()
def onchange_asize(val):
      global val_asize
      val_asize = val
      onchange_any()

def onchange_any():
      global sudoku_bw
      global thr_min_track_name,thr_max_track_name,asize_track_name
      global window_name
      global val_thr_min, val_thr_max, val_asize
      realval_thr_min = val_thr_min
      realval_thr_max = val_thr_max
      realval_asize = round(2*val_asize+3)
      sudoku_canny_tmp=cv.Canny(sudoku_bw, threshold1=realval_thr_min,
               threshold2=realval_thr_max, apertureSize=realval_asize)
      cv.imshow(window_name, sudoku_canny_tmp)

thr_min_track_name="Low TH"
thr_max_track_name="High TH"
asize_track_name="Aperture"
window_name="Canny"
cv.namedWindow(window_name)
cv.createTrackbar(thr_min_track_name,window_name,8,1024,onchange_min)
cv.createTrackbar(thr_max_track_name,window_name,43,1024,onchange_max)
cv.createTrackbar(asize_track_name,window_name,0,2,onchange_asize)

cv.waitKey()
cv.destroyAllWindows()
```

Play with it…

5. Hough transform

A standard procedure after edge detection is Hough transform in order to find out the lines limiting the objects in the image (`cv.HoughLines()`).

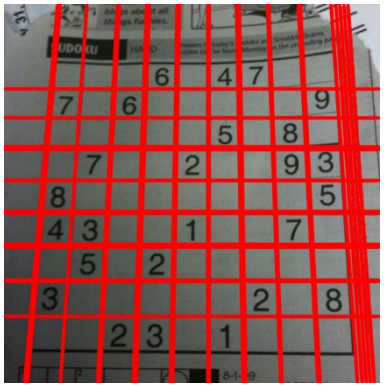
Apply Hough transform on the edge image obtained in the previous section (resolution of 1 pixel for ρ and 1°, specified in radians, for θ).

Once the lines in the image are calculated further steps are needed to paint the lines. The output variable of `cv.HoughLines` is an array of shape Nx1x2 where N is the number of lines($1 \leq i \leq N$). Two elements are given for each line (third dimension), the first dimension is ρ (`lines[i,0,0]`)

while the second one represents θ (`lines[i,0,1]`). The command `cv.line()` permits to paint a line on the input image provided the initial and final points are given

By means of a *for* loop paint the lines obtained (you can obtain the points from ρ and θ values).

TIP	*It is preferable to create a copy of the original image in which you can do the drawings*
TIP	*Write the pseudocode for painting the lines in a piece of paper and ask for help in the python implementation stage if needed*



Hough transform results

Hough transform can be also applied to detect circles using the function `cv.HoughCircles()`. The number of parameters of the transform in this case (3) is larger than for lines (2) since more parameters are required to define a circle. Additionally, Hough transform can be applied on gray level images.

Open *big_eye_glints.jpg*, convert it to gray level and find the circles containing the pupil and the eight corneal glints.

The following values are proposed (param1=100, param2=30, minRadius=50, maxRadius=60) and (param1=100, param2=13, minRadius=1, maxRadius=5) for pupil and glints detection respectively.

Once you get an acceptable detection for the eye image you can vary these numbers to evaluate the sensitivity of the Hough transform.

The output is a vector in which each circle is coded using three elements (centerx=circles[i,0,0],centery=circles[i,0,1],radius=circles[i,0,2]).

Write a python function named as *show_hough_circles* with the following input arguments. You need to find out the OpenCV function to draw circles.

show_hough_circles (dst, circles, color)

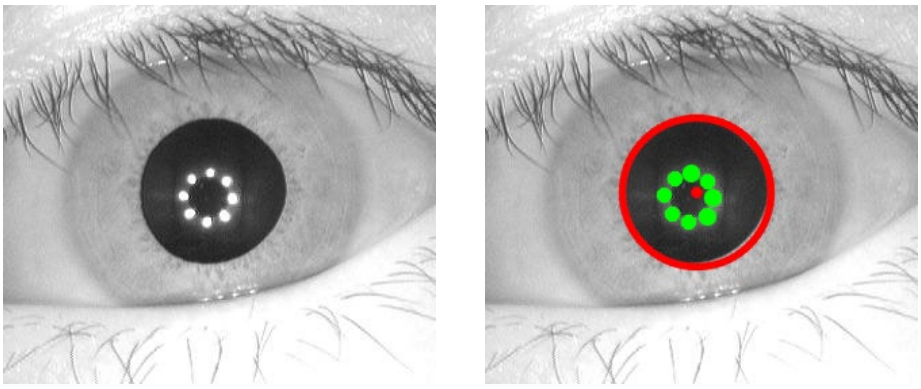
dst: image which circles are drawn on

circles: circles obtained from hough transform

color: color coded as (B,G,R) for the circles

TIP	*It is preferable to create a copy of the original image in which you can do the drawings*

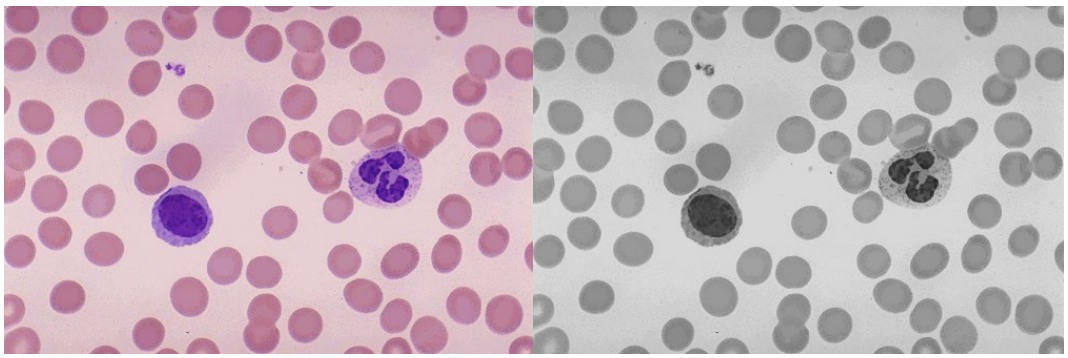
TIP	*Preprocess the output of HoughCircles:*
	`circles=np.squeeze(circles,axis=0)`
	`circles=np.round(circles).astype("int")`



Original image and Hough transform results

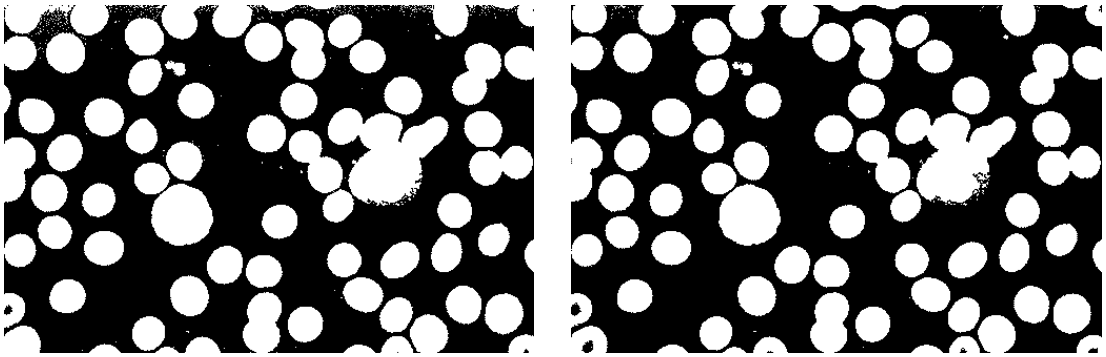
6. Watershed transform

Open image *malaria4.png* and convert it to gray level.



Original image and gray level

Try to threshold it, being the objective to differentiate the blobs representing cells from the background. You can use standard and Otsu methods. What is the optimal threshold value proposed by Otsu?

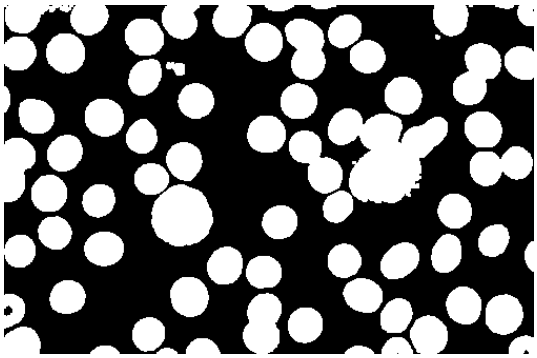


Standard segmentation (TH=185) and Otsu (TH=175)

We also recommend you create a trackbar in which you can vary the threshold value and visualize the result (like Canny example in previous sections). You can play with the trackbar to evaluate what is the best result you can get.

Once you get the best segmentation, try to "clean" it, e.g. using morphological operators such as closing, opening, hole filling and border clearing among others.

TIP	*Revisit lab session 3… you already did it!*



Result after segmentation and cleaning (you should get something similar)

The objective is to count the number of segmented cells/blobs. To this end you can use `cv.connectedComponents()`.

TIP	*Again, revisit lab session 3 to count and visualize the connected components*

How many objects have been identified in the image? Take into account that the background is also considered as an independent object.

Do you get a good segmentation?

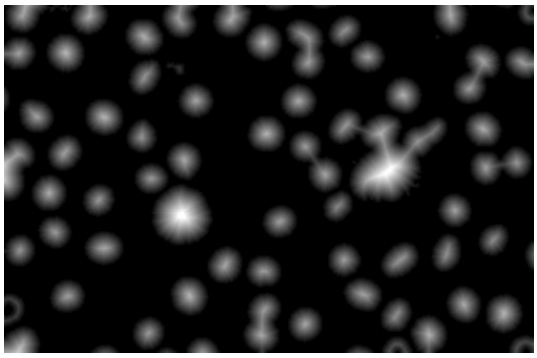
If our objective is to count the number of cells this segmentation method is not reliable even if the thresholding is correct. The fact that the blobs are together does not facilitate to identify them as separate objects with confidence.

Distance transform is an alternative. Distance transform creates an image assigning to each pixel the shortest distance to a zero value pixel. Thus, pixels closer to the background present lower values.

Apply distance transform (`cv.distanceTransform()`) to your best segmentation (segmentation+morphology) image.

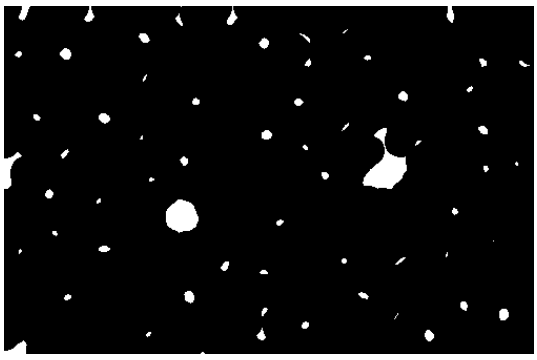
In order to show the distance transform result, take into account that the range of values of the transform present too dark values to be visually appreciated. In order to make differences visible normalize this image:

```
dt_norm_img = cv.normalize(dt_img,None,alpha=0,beta=255,
              norm_type=cv.NORM_MINMAX,dtype=cv.CV_8U)
```



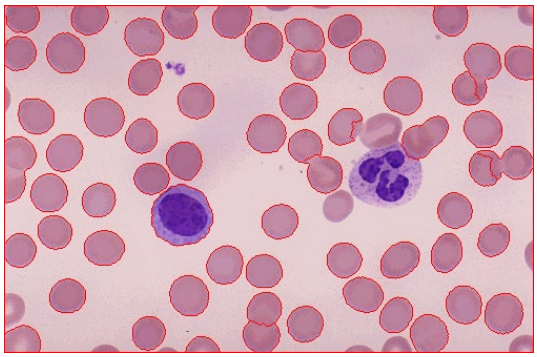
Distance transform

Once the distance transform is calculated, you can easily threshold the image (gravity center of the blobs present larger gray values) and be able to count the cells using `cv.connectedComponents()`.



Distance transform thresholded

These algorithms are sufficient if we want to find out how many cells are found in the image. However, they do not permit a whole segmentation of the pixels belonging to each cells. Watersheds algorithm is probably a much better and simpler approach to achieve our objective. However, our work till now was not in vain. Have a look to `cv.watershed()` and perform the segmentation of the image.



Watershed segmentation result

7. Advanced exercise: Principal Component Analysis

Open image *pca_test1.jpg* and convert it to gray level. Threshold it.

OpenCV provides an easy way to segment images using `cv.findContours()`. For this type of simple images it works pretty well (`mode=cv.RETR_LIST` and `method=cv.CHAIN_APPROX_NONE` work nice)

Additionally, `cv.drawContours()` permit to overlay contours in the image.

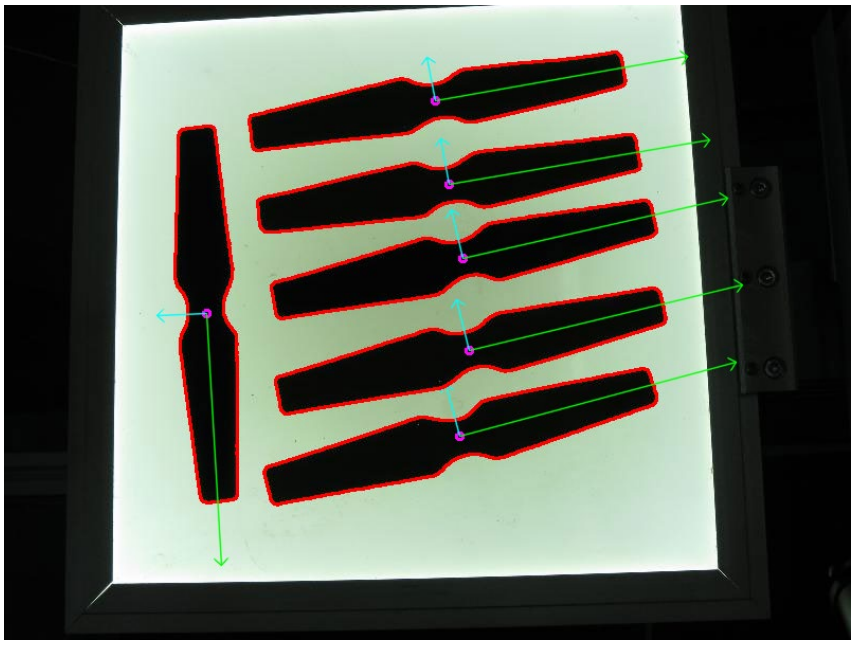
Once you understand how to use `cv.drawContours()` implement a *for* loop to overlay all of them.

As result of the *for* loop you can observe that apart from the contours of the objects we are interested in, other contours are highlighted.

Modify the *for* loop so that only the contours of the blades are shown. You can check the properties of the output argument of `cv.findContours()` and use them to discriminate the contours you want to draw such as `cv.contourArea()`.

Finally, as a further analysis step we can think about an application in which the orientation of the objects is required in order to take decisions. PCA transform permits to calculate the main axes of an object, i.e. eigenaxes. The command `cv.PCACompute2()` permits to calculate the PCA transform for each object (in this case using the contour points as the sample).

TIP	*It is preferable to create a copy of the original image in which you can do the drawings*



Example of PCA applied to contours

```
def getOrientation(pts,img):
#Construct a buffer used by the pca analysis
      sz = len(pts)
      data_pts = np.empty((sz,2),dtype=np.float64)
      for i in range(data_pts.shape[0]):
            data_pts[i,0] = pts[i,0,0]
            data_pts[i,1] = pts[i,0,1]

      #Perform PCA analysis
      # Complete here the function with two additional lines
      # performing the actual PCA analysis

      #Store the center of the object
      cntr = (int(mean[0,0]),int(mean[0,1]))

      #[visualization]
      # Draw the principal components
      cv.circle(img,cntr,3,(255,0,255),2)
      p1 = (cntr[0]+0.02*eigenvectors[0,0]*eigenvalues[0,0],
            cntr[1]+0.02*eigenvectors[0,1]*eigenvalues[0,0])

      p2 = (cntr[0]-0.02*eigenvectors[1,0]*eigenvalues[1,0],
            cntr[1]-0.02*eigenvectors[1,1]*eigenvalues[1,0])

      drawAxis(img,cntr,p1,(0,255,0),1)
      drawAxis(img,cntr,p2,(255,255,0),5)

      #orientation in radians
      angle = np.arctan2(eigenvectors[0,1],eigenvectors[0,0])

      return angle
```

```
def drawAxis(img, p_,q_, colour,scale):
      p = list(p_)
      q = list(q_)
      #[visualization1]
      angle = np.arctan2(p[1]-q[1],p[0]-q[0]) #angle in radians
      hypotenuse = np.sqrt((p[1]-q[1])*(p[1]-q[1])+
                  (p[0]-q[0])*(p[0]-q[0]))

      #Here we lengthen the arrow by a factor of scale
      q[0] = p[0]-scale*hypotenuse*np.cos(angle)
      q[1] = p[1]-scale*hypotenuse*np.sin(angle)
      cv.line(img,(int(p[0]),int(p[1])),
             (int(q[0]),int(q[1])),colour,1,cv.LINE_AA)

      #create the arrow hooks
      p[0] = q[0]+9*np.cos(angle+np.pi/4)
      p[1] = q[1]+9*np.sin(angle+np.pi/4)
      cv.line(img,(int(p[0]),int(p[1])),
             (int(q[0]),int(q[1])),colour,1,cv.LINE_AA)

      p[0] = q[0]+9*np.cos(angle-np.pi/4)
      p[1] = q[1]+9*np.sin(angle-np.pi/4)
      cv.line(img,(int(p[0]),int(p[1])),
             (int(q[0]),int(q[1])),colour,1,cv.LINE_AA)
```

LAB SESSION 5
DEEP LEARNING

Please, consult the companion Jupyter notebook, containing the guides for this lab session.

LAB SESSION P1
Project: preliminaries

In this session, we will introduce some practical tools that can be used during the development of your project.

1. Constructing a python library

Every time you perform an `import` in your code you are importing a python library. A python library is a .py file in which several functions are saved.

Once you `import` the library, you can call any function contained in the corresponding .py.

Construct a library named as `upna_utils.py` in which the following (already used) functions are saved:

```
open_reconstruction(image, se, n)
close_reconstruction(image, se, n)
basic_reconstruction(marker, mask, se, method)
geo_step(marker, mask, se, method)
imfill(image)
imclearborder(image)
ShowConnectedComponents(labels_im)
```

Save this library file in the same folder of your code.

2. Data structures in Python

Creating variables combining different types of information (numbers, text, etc) is simple in Python. Python allows for this under the data type *dictionaries*. Execute the following piece of code:

```
blob1 = {'Area':4, 'Name':'Circle', 'Perimeter':11}
blob2 = {'Area':7, 'Name':'Square', 'Perimeter':55}
print(blob1['Perimeter'])
print(blob2['Name'])
```

`blob1` and `blob2` are variables having three fields: `Area`, `Name` and `Perimeter`. As you noticed `Area` and `Perimeter` are numeric fields while `Name` is text.

You can add elements to a previously defined dictionary:

```
blob1['std_dev'] = 24
```

We can also construct lists of dictionaries. See the following example:

```
blobs=[]
blobs.append(blob1)
blobs.append(blob2)
print(blobs[1])
print(blobs[1]['Area'])
```

3. Advanced indexing

Here are some advanced indexing examples that may be useful during the development of the project:

```
import numpy as np
test_array = np.array([0,0,1,1,1,0,0,2,2,0,0,3,3,3,3])
```

First option: automatic casting.

```
out_list_1 = 1*(test_array==1)
print(out_list_1)
```

Compare the next two outputs:

```
out_list_1_1 = (test_array>=2)
print(out_list_1_1)
print(1*out_list_1_1)
```

Second option: `where` function from numpy library.

```
out_list_2 = np.where(test_array==2,1,0)
print(out_list_2)
```

Third option: logical indexing.

```
out_list_3 = 0*test_array
out_list_3[test_array==3] = 1
print(out_list_3)
```

Or with a more elaborated logical condition:

```
out_list_3_1 = 0*test_array
out_list_3_1[np.bitwise_or(test_array==2,test_array==3)] = 1
print(out_list_3_1)
```

4. Measuring execution time

The execution time of an algorithm is key for many real time processes or to calculate the computational resources needed in order to get a result in a reasonable time.

Python provides an easy way to measure the execution time for specific pieces of your code. The library to be imported is `time`.

The following lines of codes permit to measure the elapsed time between two points of the execution.

```
import time
#Measuring time...
t=time.process_time()

#do some stuff
for i in range(1000000):
      kk=np.log10(i+1)

elapsed_time = time.process_time() - t
print(elapsed_time)
```

There is also a more sophisticated and precise tool to evaluate execution times in your code: the standard library *cProfile*. This library provides deterministic profiling of Python code informing you with complete statistics about your functions: number of calls, total execution time and many other details. You can read more about this library here.

5. Extracting characteristics from blobs

Several times the objective of your analysis is to extract some numerical or qualitative evaluation of the image to take a decision. The visual evaluation is normally not feasible and frequently subjective. Once you finish your segmentation the usual way to work is to translate the segmentation into a numerical value in which the result of your segmentation is summarized.

OpenCV provides several tools to extract characteristics from your blobs. Open *shapes_and_colors_trim.png* and convert it into gray level. Threshold it using 80 as threshold value.

Import your `upna_utils.py` library and clean your segmentation by means of a border clearing procedure, hole filling… Extract connected components in the image. By means of a for loop visualize the different connected components in the image. In each iteration an isolated connected component is to be shown.

The following functions permit extracting useful characteristics of your segmented objects. Have a look at the information they provide.

```
cv.findContours()
cv.moments()
cv.contourArea()
cv.arcLength()
cv.boundingRect()
```

Once your image is segmented, it can be useful to perform drawings in the images or to add some text for annotation purposes. You already know how to sketch lines and circles but there are more…

`cv.drawMarker()` permits to add a varying shape marker in a specific pixel of your image. Paint a cross in the center point of the image.

Using `cv.rectangle()` draw a green rectangle in position (20, 30) having a width value of 100 pixels and height value of 40 pixels.

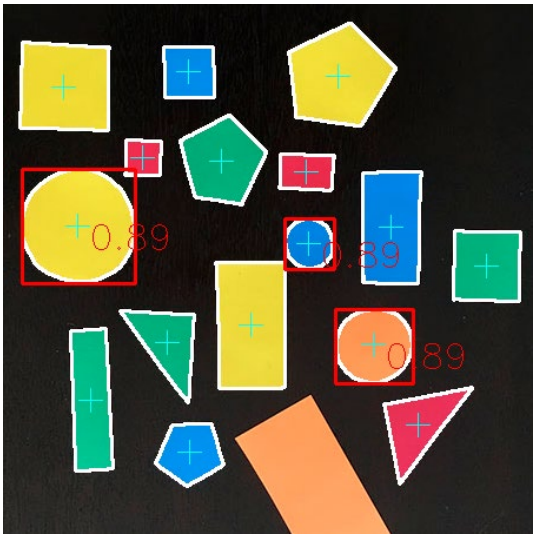
`cv.putText()` permits to annotate the image by placing formatted text in the image. Close to the marker you did by using `cv.drawMarker()` add a text showing the position (row, col) of that point in blue using HERSHEY_SIMPLEX font and thickness = 2.

`cv.drawContour()`permits to draw a contour (contour is a specific OpenCV object)

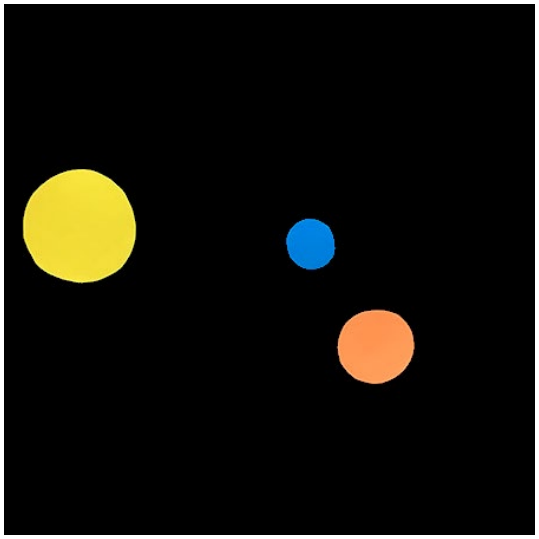
Extract the contours of the clean image and draw them on the original image (in green).

For each contour calculate its centroid and circularity ($4\,\pi\,area/perimeter^2$) value. Draw in the original image, a cross marker in the centroid of each object (using cyan color). Additionally, the contours of the objects need to be drawn. Detect those objects having a circularity higher than 0.85. For those objects, include in the image the corresponding bounding boxes using red color and the circularity value.

Finally, create a new image in which those objects with a circularity value higher than 0.85 are exclusively shown.



Original image adding contours, markers, and circularity values



Final result

TIP	*You can apply masks (logical operator and) using OpenCV.* `cv.bitwise_and()` *and* `cv.bitwise_or()` *are provided*

You can also use `connectedComponentWithStats()` function to obtain, for each label, statistics such as the area of the object. Play with this function to obtain different features of the segmented objects in the image.

6. Measuring accuracy

In order to evaluate the performance of our algorithms, different figures of merit can be used. In the following lines some important accuracy measures will be explained.

Results may be measured in terms of positive and negative detection rates.

TP: true positive. The number of cases in which your algorithm predicts the case as positive and the classification is correct (ground truth=positive/true/1).

TN: true negative. The number of cases in which your algorithm predicts the case as negative and the classification is correct (ground truth=negative/false/0).

FP: false positive. The number of cases in which your algorithm predicts the case as positive and the classification is not correct (ground truth=negative/false/0).

FN: false negative. The number of cases in which your algorithm predicts the case as negative and the classification is not correct (ground truth=positive/true/1).

Confusion matrices are of common use to show the rate of positive and negative detections.

		Actual class	
		POSITIVE	**NEGATIVE**
Predicted class	**POSITIVE**	*TP*	*FP*
	NEGATIVE	*FN*	*TN*

Table 1.- Confusion matrix

Sensitivity (true positive rate) is the number of positive cases correctly detected with respect to the total of positives in the sample.

$$sens = \frac{TP}{(TP + FN)}$$

Specificity (true negative rate) is the number of negative cases correctly detected with respect to the total of negatives in the sample.

$$spec = \frac{TN}{(TN + FP)}$$

Accuracy is the number of correctly detected cases (positive+negative) with respect to the total of cases.

$$acc = \frac{(TP + TN)}{(TP + TN + FP + FN)}$$

In image segmentation applications, one of the most employed metrics is the *Dice* coefficient (DC) or F_1 score to calculate the overlapping ratio between the correct and estimated segmentations. Dice coefficient is calculated as:

$$DC = F_1 = \frac{2\,|A \cap B|}{|A| + |B|}$$

where $|A|$ and $|B|$ represent the number of object pixels in set A (ground truth segmentation) and B (estimated segmentation), respectively, and $|A \cap B|$ is the number of pixels in the intersection of these two sets.

Jaccard coefficient (JC) is also intuitive and is calculated as:

$$JC = \frac{|A \cap B|}{|A \cup B|}$$

also named as the intersection over union.

More details about overlapping estimation can be found here.

Also *Dice* coefficient can be calculated as:

$$DC = \frac{2\,TP}{(2\,TP + FP + FN)} = 2\,\frac{acc\ sens\ -\ sens\ spec}{sens\ +\ acc\ sens\ -\ spec\ +\ acc\ spec\ -\ 2\,sens\ spec}$$

Open images *tptnfpfn.png* and *tptnfpfn_gt.png.* These images simulate a ground truth image and a segmentation result. Calculate the number of TP, TN, FP and FN.

 **TIP**	*Check the OpenCV function* `countNonZero()`

LAB SESSIONS P2-P5
Segmentation of CD8+ lymphocytes from microscopy images of urothelial carcinoma

1. Introduction

Clinical Framework

Urothelial carcinoma, which is predominantly seen in men, also known as transitional cell carcinoma, is a type of cancer that begins in urothelial cells. The term "transitional cells" is used for urothelial cells because they change shape; these cells present flexibility, stretching when the bladder is filled with urine and contracting when it is emptied. Urothelial cells cover various structures, including the urethra, bladder, ureters, renal pelvis, and certain other organs (see Figure 1).

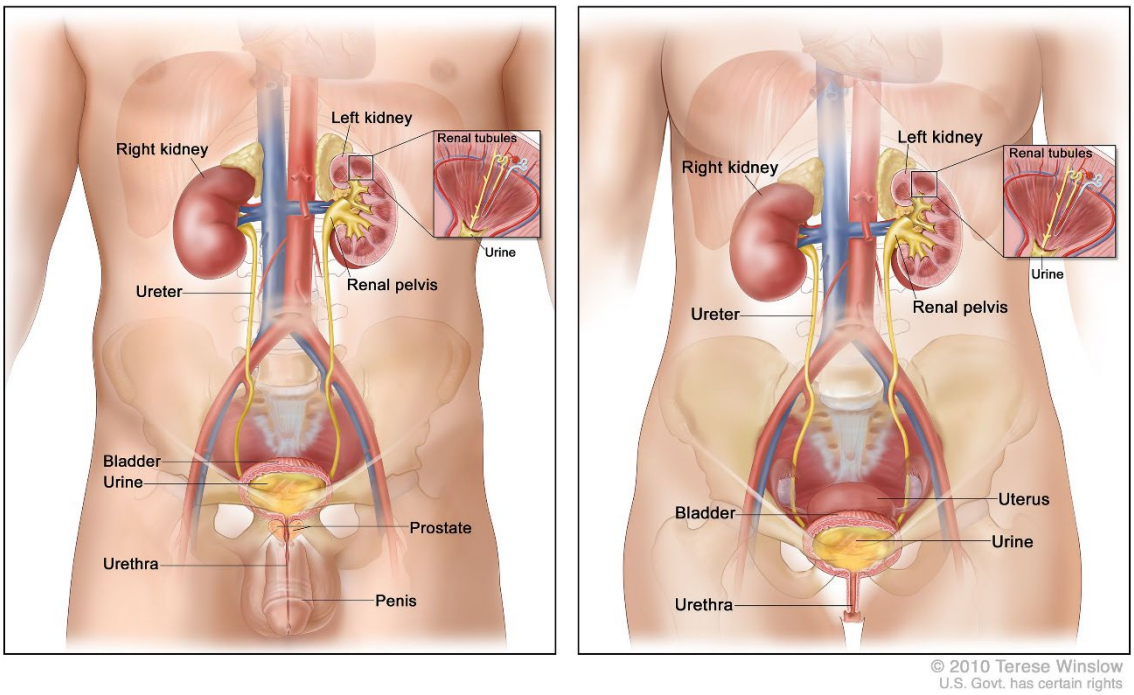


Figure 1: Anatomy of the male urinary system (left panel) and female urinary system (right panel), illustrating the kidneys, ureters, bladder, urethra, prostate, and penis (left panel), as well as the uterus (right panel). Extracted from [1].

Almost all bladder cancers fall under the category of urothelial carcinomas, making them the 10th most commonly diagnosed cancer and the 13th leading cause of cancer deaths worldwide. Bladder cancer is an aggressive malignancy with a five-year survival rate of about 5% in the metastatic cases [2].

Tumoral and immune cells coexist within what is called the *tumor microenvironment*. The interactions between these cells, within the tumor itself or in its surrounding stroma, are crucial for patient diagnosis. Currently, several studies have been conducted to identify patterns in

cellular interactions specific to each type of cancer, aiming to enhance the understanding of the disease. Indeed, this approach enables the development of more individualized treatments for each type of cancer. The tumor immune microenvironment has a direct impact on cancer progression, metastasis probability and response to cancer treatment, such as immunotherapy, being a pillar in the hallmarks of cancer [3].

Lymphocytes, or T cells, are cells within the immune system that act as cell mediators with specific functions depending on their type. The most common types include CD8+ T cells, CD4+ T cells and T regulatory cells. In this project, the focus will be on CD8+ T cells, which are pathogens killers, because they play a vital role in anticancer treatment as key immune cells for killing cancer cells [4].

The success of the proposed biological image analysis task in this project heavily relies on the accurate detection and segmentation of cells and nuclei from fluorescence microscopy images [5]. Fluorescent microscopy allows the simultaneous visualization of different fluorescent markers. Each fluorochrome has a characteristic, band-limited, emission spectrum, and thus fluorescence microscopes make use of emission filters to capture and separate the light that corresponds to each marker. This way, different proteins that define different cell types can be simultaneously observed, along with more general markers, such as DAPI staining that binds with the DNA and captures the cell nuclei shapes. A fluorescence image can thus be composed of several channels, each channel corresponding to a specific emission spectral band, and hence a specific fluorescent marker. In our project, each tumor microenvironment image is composed of two channels: DAPI channel, which identifies all cell nuclei, and CD8 channel, which contains a cell membrane protein marker that helps identifying CD8+ lymphocytes.

Figure 2 shows an example of a fluorescence image of urothelial carcinoma, with the two channels visualized with pseudocolors: DAPI staining in blue, and CD8 signal in red.

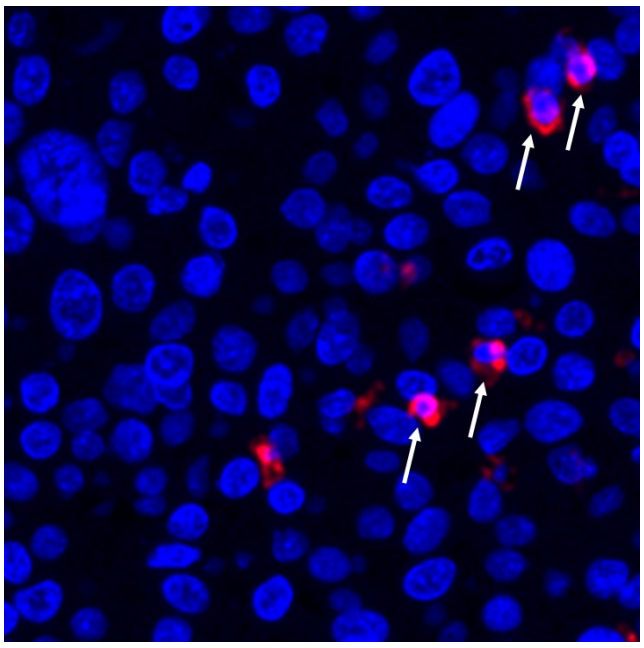


Figure 2.- Fluorescence microscopy image of the tumor microenvironment of urothelial carcinoma. DAPI staining of cell nuclei is shown in blue, and CD8+ regions in red. CD8+ lymphocytes are identified with white arrows.

Image Acquisition

The fluorescence images available for this project come from two tissue microarrays (TMAs) of biopsy samples of urothelial carcinoma patients. The tissue microarray technique organizes several small, circular biological samples of tissue, called *cores*, in a single recipient, called *microarray*. This method introduced significant advantages, such as allowing to generate more data from a single sample, analysing large number of data simultaneously, presenting experimental uniformity by treating each core in an identical manner or decreasing the volume, cost and time of the assay.

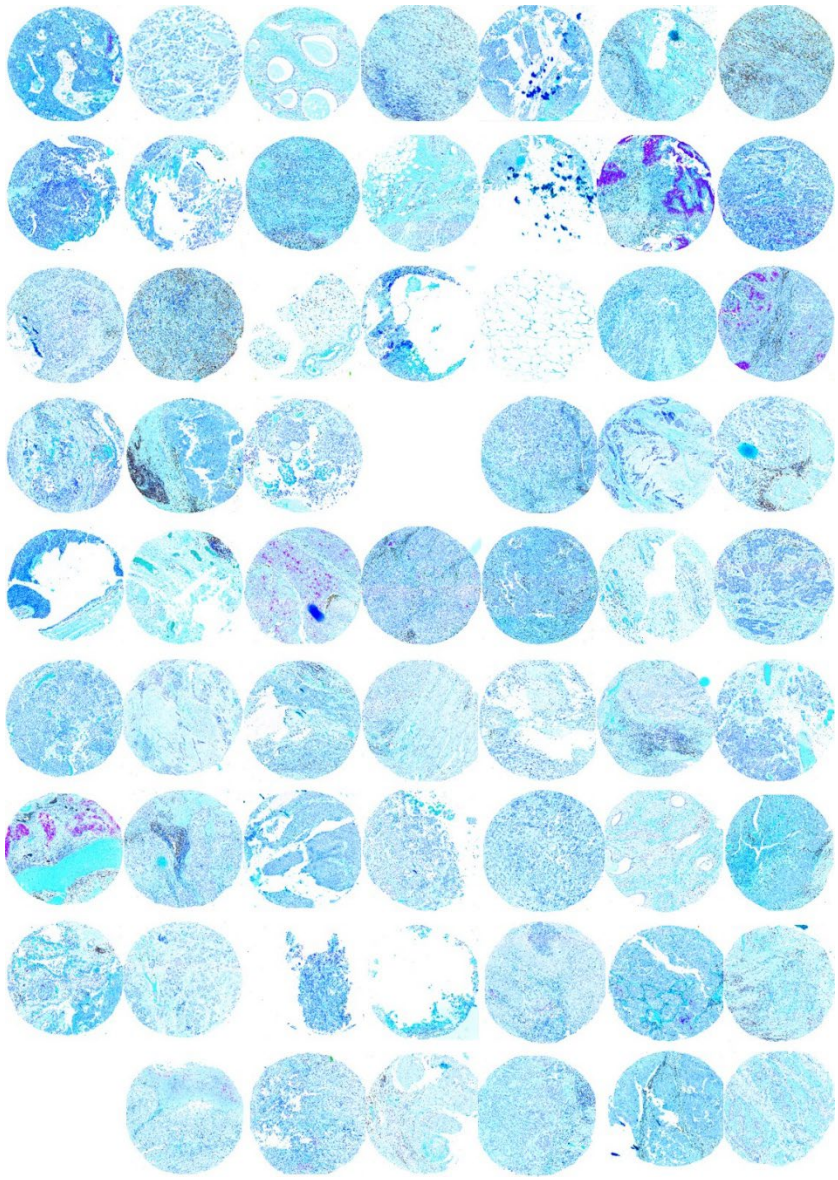


Figure 3: Distribution of biopsy sample cores in a TMA.

2. Objectives

The objectives of the project can be divided into three parts:

1. To implement a segmentation algorithm for cell nuclei in microscopy images. The result of the segmentation will be a labelled image in which each different nucleus is tagged with a unique number.

2. To implement a CD8+ T-cell detection algorithm in microscopy images. The result of the detection will be the number of CD8+ T-cells in the image.
3. To evaluate the performance of the implemented system in terms of dice coefficient, detection rate, and execution time.

One of the key elements of the project is the dataset provided. The CD8+ T-cell dataset is composed by a set of real images and the manual annotation of these images (ground truth). We would like to thank Dr. Carlos de Andrea and María Sangüesa for the basis of this material.

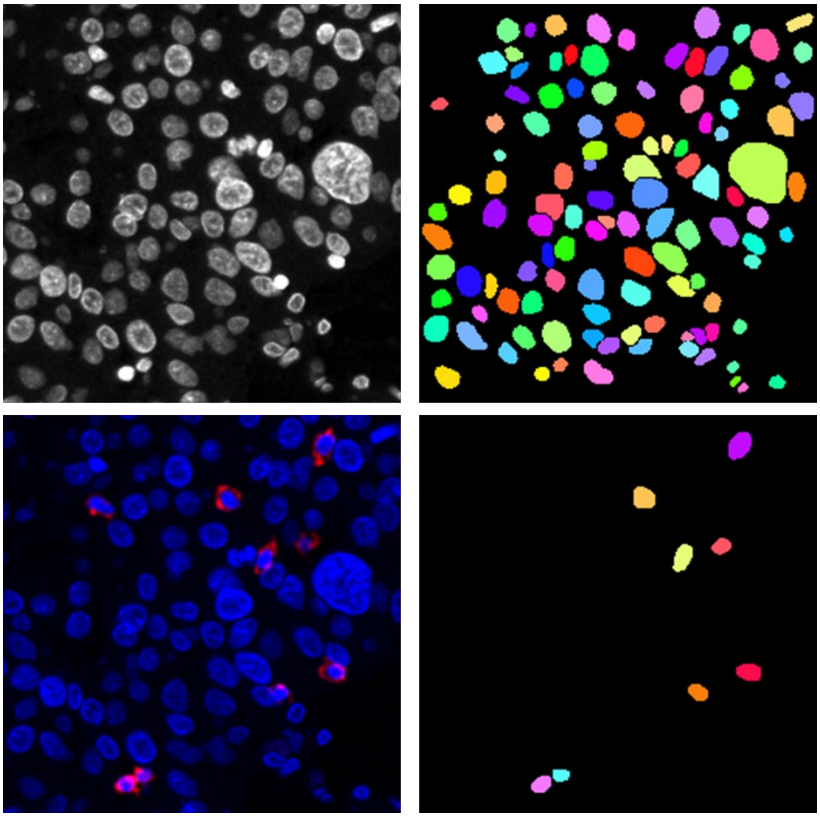


Figure 4.- DAPI image showing cell nuclei (top left) and its corresponding ground-truth labelmap (top right) shown with a random colormap. RGB image showing DAPI (in blue) and CD8 (in red) channel images fused (bottom left), and its corresponding ground-truth labelmap of CD8+ T-cells (bottom right).

3. Dataset

For the completion of this project, we will have four sets of images:

- ✓ Images of DAPI signal for nuclei segmentation: we have 21 images of a fixed size of 256x256 pixels, obtained as crops from larger images of complete cores of a tissue micro-array (TMA). The image names follow the convention *cutout_XX.tif*, where XX indicates the sequential numbering of the crop.
- ✓ Ground-truth (GT) for nuclei segmentation: this set contains the reference segmentations of cell nuclei. These annotations have been created based on the results obtained using a deep-learning-based pretrained nuclear segmentation algorithm, and subsequently edited manually until an acceptable final result was achieved. The editing criteria were supervised by a pathologist with over 15 years of experience in the analysis of such images. We will also refer to these images as nuclear labelmaps, indicating that we are associating a unique label (a consecutive

integer) with each cell nucleus in the image. Label 0 will always correspond to the background of the image

- ✓ Images of CD8 signal for the detection of CD8+ T-cells: this is the same set of 21 images, in this case showing the signal corresponding to CD8. The image names follow the same convention as the DAPI ones. **Note: DAPI and CD8 are actually two fluorescent channels corresponding to the same field of view, i.e., they compose a 2-channel fluorescence image.**
- ✓ Ground-truth (GT) for CD8+ T-cell detection: this set contains the reference segmentations of CD8+ T-cells. These annotations have been manually created and supervised by a pathologist with over 15 years of experience in the analysis of such images. We will also refer to these images as CD8+ T-cell labelmaps, indicating that we are associating a unique label (a consecutive integer) with each CD8+ T-cell in the image. Label 0 will always correspond to the background of the image.
- ✓ You will also find a folder with RGB images of the fusion of both channels (DAPI and CD8). These images are just meant to be used as visual help, **no image processing should be carried out with them**.

4. Deliverables

You can create as many functions and code as needed to develop your project.

You are required to supply all the files that are needed to run and evaluate your segmentation system.

However, the following functions and documents are a requisite of the project and they must be employed by your quantification system (more details in the following sections). The functions employed by your code will be contained in a library (`lymphocytes.py`) that you should import into your `main.py` code.

The deliverables of your project are:

Library: lymphocytes.py

The library `lymphocytes.py` will contain, at least, the following functions.

function `nuclei_labelmap = Seg_Cell_Nuclei(img_DAPI, params_seg)`

This function will be responsible for performing the segmentation of the cell nuclei in the input image of the DAPI signal. Based on this input image and the parameter values passed as the second argument to this function, it will calculate the labelmap that identifies each of the cell nuclei in the image with a unique integer number.

Input arguments:

- ✓ `img_DAPI`: is the gray level image, corresponding to the DAPI signal, to be segmented, as shown in Figure 4 (top left)
- ✓ `params_seg`: it is a dictionary containing the parameters used by your segmentation algorithm. The data type is a `dictionary`. This input is optional, and its content is free

Output arguments:

- ✓ `nuclei_labelmap`: is the output nuclei labelmap, as shown in Figure 4 (top right)

function `n_CD8cells = Det_CD8_Cells(img_CD8, nuclei_labelmap, params_det)`

This function will be responsible for performing the detection of the CD8+ T-cells in each image field. Based on the image of the CD8 signal, the labelmap of segmented cell nuclei, and the parameter values passed as the third argument to this function, it will calculate the number of CD8+ T-cells in the image.

Input arguments:

- ✓ `img_CD8`: is the gray level image ,corresponding to the CD8 signal, that will be used to determine which cell nuclei correspond to CD8+ T-cells
- ✓ `nuclei_labelmap`: is the output of the previous function, containing a labelmap with the nuclei segmentation
- ✓ `params_det`: it is a dictionary containing the parameters used by your detection algorithm. The data type is a `dictionary`. This input is optional, and its content is free

Output arguments:

- ✓ `n_CD8cells`: is the number of CD8+ T-cells in the image.

function `general_results = Analyze_Set(general_configuration)`

This function implements the highest-level processing. Starting from the general configuration data (such as paths to the directories of images to be segmented, GT path, etc.), it will perform the segmentation and detection tasks, returning an output list containing all the obtained data.

Input arguments:

- ✓ `general_configuration`: is a dictionary containing the parameters used by your quantification algorithm. It contains **at least** the following fields:
  - `imgDAPI_path`: is the path to the folder containing the images of the DAPI nuclear signal
  - `gtNuclei_path`: is the path to the folder containing the GT labelmaps of cell nuclei
  - `imgCD8_path`: is the path to the folder containing the images of the CD8 signal
  - `gtCD8_path`: is the path to the folder containing the CD8+ T-cell labelmaps
  - `out_path`: is the path to the folder where all partial results considered necessary for subsequent analysis will be stored. Specifically, the final nuclei labelmaps for each case will be saved in this folder

  The variables `params_seg` and `params_det` mentioned as inputs in the previously described functions will typically be subsets of this variable `general_configuration`.

Output arguments:

- ✓ `general_results`: is a list of dictionaries. Each element of this list must have **at least** the following fields:
  - `image_ID`: a string identifying the corresponding image (cutout_XX)
  - `total_dice`: the numerical value of the Dice coefficient, obtained from the comparison of the nuclear segmentation labelmap with the GT

labelmap. Use the provided function (`Evaluate_Labelmap()`) to calculate this value

- `det_error`: numerical value of the error in the detection of CD8+ T-cells. Use the provided function (`Evaluate_Detection()`) to calculate this value
- `exec_time`: is the execution time needed to analyze each case

⚠ **IMPORTANT**	*No image processing should be carried out in this function* *The fields* `imgDAPI_path`, `gtNuclei_path`, `imgCD8_path` *and* `gtCD8_path` *contain a single string each, not a list of strings* *It is mandatory that the data type of* `general_configuration` *be a dictionary*
⚠ **IMPORTANT**	*This function will not be evaluated if any of these requirements are ignored.*

In this library you also must include these **supplied** functions:

function `mean_dice = Evaluate_Segmentation(labelmap_test, labelmap_gt)`

This function evaluates the performance obtained by your algorithm in the nuclear segmentation, comparing the proposed labelmap with its corresponding ground truth. The output value is obtained as a mean value from all dice values for each nucleus present in the particular image.

Input arguments:

- ✓ `labelmap_test`: segmentation labelmap to be evaluated against ground truth.
- ✓ `labelmap_gt`: ground truth labelmap used as reference.

Output arguments:

- ✓ `mean_dice`: the global weighted result of dice coefficient, calculated from particular dice coefficients for each cell nucleus.

function `det_err = Evaluate_Detection(n_CD8cells, labelmap_gt_CD8)`

This function evaluates the performance obtained by your algorithm in the detection of CD8+ T-cells, comparing the proposed number of CD8+ T-cells in the image with its corresponding ground truth. This ground truth is the labelmap of CD8+ T-cells, from which the correct number of detections will be obtained. The output detection error is obtained as:

$$detErr = |N_{GT} - N_{det}|$$

where N_{GT} is the correct number of CD8+ T-cells, N_{det} is the output number of detections given by the proposed algorithm, and $detErr$ is the output detection error.

Input arguments:

- ✓ `n_CD8cells`: number of detections given by the proposed algorithm.
- ✓ `labelmap_gt_CD8`: ground truth labelmap of CD8+ T-cells used as reference.

Output arguments:

- ✓ `det_err`: the detection error.

function `fus_img, key_pressed = Fuse_Labelmap(base_img, labelmap, false_color_list, alpha, showQ, window_title)`

Function to fuse a base image with its correspondence labelmap. The fusion technique will be based on the variation of the transparency of the labelmap over the base image. It returns the fused image and the key pressed to close the window.

Input arguments:

- ✓ `base_img`: MxN gray level input image that serves as background of the final fused result. In this project, it will be the DAPI signal image.
- ✓ `labelmap`: MxN input labelmap to be overlaid with the previous image. It is obtained from `Seg_Cell_Nuclei()` function
- ✓ `false_color_list`: list of false colors used for coloring the input labelmap. It is obtained directly from `Create_List_False_Color()` function, included with the rest of the supplied functions.
- ✓ `alpha`: value of alpha parameter controlling the transparency level of the input labelmap.
- ✓ `showQ`: boolean variable to control if a window is opened, showing the fused image, with a slider controlling the value of the parameter alpha.
- ✓ `window_title`: the title for the opened window.

Output arguments:

- ✓ `fus_img`: MxNx3 fused image, resulting of the sum between the base image and the labelmap with the specified false color.
- ✓ `key_pressed`: returns the code of the key pressed to close the window. This information can be used to halt the caller function.

⚠ **IMPORTANT**	*You can download these three functions, with their corresponding auxiliary functions, from miAularIO. You can consult with your professors any doubts about their implementation*

Optional functions

function `output_labelmap = Cleaning_Labelmap(input_labelmap, params)`

This function can be integrated within the nuclei segmentation function itself. It will be responsible for cleaning the labelmap obtained in the previous steps to obtain a labelmap closer to the ground truth (GT). Specifically, the following criteria should be taken into account:

- ✓ Nuclei that do not appear complete in the image will be omitted
- ✓ Regions that, based on their morphological characteristics (e.g., size), are not considered to be cell nuclei, will be omitted

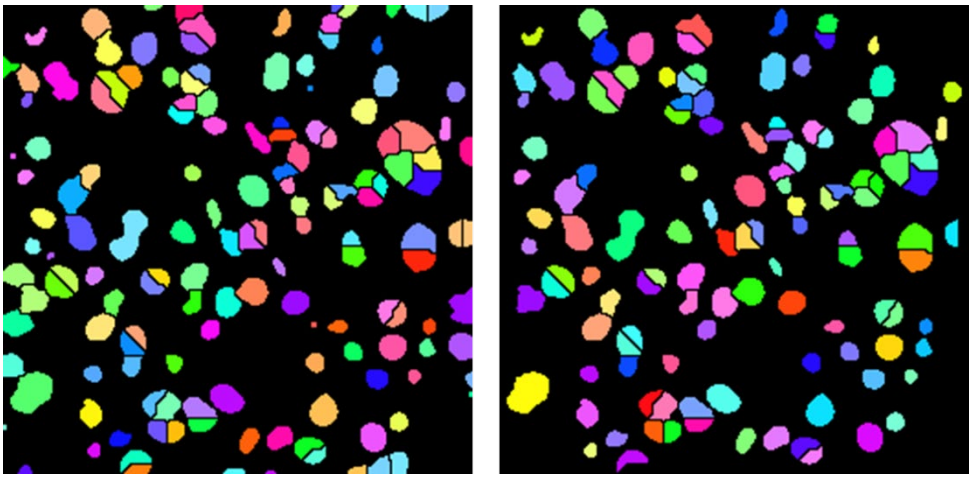


Figure 1.- Example of the cleaning process: input labelmap (left) and its corresponding cleaned one (right)

Input arguments:

- ✓ `input_labelmap`: is the input labelmap to clean
- ✓ `params`: it is a dictionary containing the parameters used by your cleaning algorithm. The data type is a `dictionary`. This input is optional, and its content is free

Output arguments:

- ✓ `output_labelmap`: is the clean output nuclei labelmap

Main script: main.py

`main.py` is the main script you used to evaluate your code. You are free to write the code using your own criteria. The library `lymphocytes.py` will be imported in the beginning of your main file.

The idea is that you can conduct a thorough study of the performance of your code.

Potential questions to be answered in your main code would be:

- What are the parameter values in your process that are key for an accurate detection of CD8+ T-cells?
- What is the best global mean dice coefficient you obtain for the nuclear segmentation?
- How sensitive is the dice coefficient to the alternative parameters used by your algorithm?
- What is the lowest CD8+ T-cell detection error you obtain?

Requirements.txt

Containing the libraries needed to install in the environment in order to run your code.

Report

A report with a maximum of 10 pages (pdf). The report should at least have a section of introduction, a section of objectives, a section containing the method, a section of results, a critical discussion, and a final section with conclusions. Pay special attention to the methods, results, and discussion sections. Performance values (detection error and dice coefficient) should be clearly stated for every setup that you evaluated. Please, be as clear as possible and

use as many tools as you can, -boxplots can be a helpful tool- (graphs, intermediate result images, etc.) to support your conclusions. Reports with python code will not be accepted.

5. Materials

The following materials are available in miAulario to develop your project:

- Dataset containing the 21 cases for nuclear segmentation and CD8+ T-cell detection, with the corresponding ground truth labelmaps
- Pre-implemented functions and their auxiliary ones:
  - `Evaluate_Segmentation.py`
  - `Evaluate_Detection.py`
  - `Fuse_Labelmap.py`

6. Grading Policy

Your grade will be based on the following criteria:

✓ Dice results of nuclear segmentation (20%), obtained from:

$$min\left(10, max\left(0, 10\frac{x-0.5}{0.80-0.5}\right)\right)$$

where x is the average value of the 21 dice coefficients obtained from the output of the `Analyze_Set()` function.

✓ Detection error of CD8+ T-cells (20%), obtained from:

$$min\left(10, max\left(0, 10\ -\ 10\frac{y-0.1}{0.5-0.1}\right)\right)$$

where y is the accumulated value of the 21 detection errors obtained from the output of the `Analyze_Set()` function, divided by the total number of CD8+ T-cells in the ground truth (74). 53

✓ Execution time (10%), obtained from[1]:

$$min\left(10, max\left(0, 10 -\ 10\frac{z-0.5}{2-0.5}\right)\right)$$

where z is the mean value of 8 particular execution times in seconds obtained from the output of the `Analyze_Set()` function. This time must be measured in this `Analyze_Set()` function, including exclusively each call to `Seg_Cell_Nuclei()` and `Det_CD8_Cells()` functions (including in this evaluation the optional nuclei labelmap cleaning function). Remember to disable all debug options in your code, including writing to disk auxiliary data, such as intermediate images or segmentation labelmaps, in order to get the minimum possible time

✓ Optional function (10%): implementation of the optional function, described above.
✓ Report and evaluation (main.py): 40%.
  - Report: presentation, formatting, writing style, use of scientific language, graphical support and spelling.
  - Critical discussion about the obtained results (included in the report).
  - Method: robustness, originality and coherence.

[1] The execution time will be measured using professor's computer

- Implementation: commented code, function and variables naming, efficiency, etc.

✓ Penalties:

- Installation problems: -1 pt
- Sending the environment or the dataset: -2 pts
- Not respecting the name of the functions or the format of input/output arguments.: -2 pts
- Execution problems: up to -2 pts

7. Important issues

- The project is done in groups of two.
- It is important that the format for your input and output parameters is as specified in this guide. Remember that the core functions SHOULD NOT have any other input/output parameters than those previously mentioned. Your functions will be used by our evaluation code and any changes in arguments' formats will cause the evaluation to crash.
- Remember that, in addition to the essential functions, you should provide any other functions whose implementation is necessary for the execution of your functions. Also, do not use different parameter values for different images. Exactly the same functions should be used for all the images without changing any input or parameter values.
- For the sake of fairness, we will not answer questions regarding the methods you have to implement. Studying and understanding these methods is, in fact, your responsibility.
- You may ask questions regarding English in the event that you do not understand this document, python help texts, or texts from other sources. You may also ask questions related to specific programming issues (debugging problems).
- If we feel that you are asking "too many" questions, we reserve the right to stop answering them or to limit the number of questions.
- Remember that we could choose several projects to make an oral presentation; we may be more likely to choose those students that are less active during lab sessions.
- Deadline: The deadline for receipt of your projects is **18:00 on 8 January, 2025**. You should send an email containing your code and report to rcabeza@unavarra.es.

8. References

[1] 'What Is Bladder Cancer? - NCI'. Accessed: Dec. 10, 2023. [Online]. Available: https://www.cancer.gov/types/bladder

[2] 'Cancer of the Urinary Bladder - Cancer Stat Facts', SEER. Accessed: Jan. 25, 2024. [Online]. Available: https://seer.cancer.gov/statfacts/html/urinb.html.

[3] D. S. Chen and I. Mellman, 'Elements of cancer immunity and the cancer–immune set point', Nature, vol. 541, no. 7637, Art. no. 7637, Jan. 2017, doi: 10.1038/nature21349.

[4] K. Iwahori, 'Cytotoxic CD8+ Lymphocytes in the Tumor Microenvironment', in *Tumor Microenvironment: Hematopoietic Cells – Part A*, A. Birbrair, Ed., in Advances in Experimental Medicine and Biology. , Cham: Springer International Publishing, 2020, pp. 53–62. doi: 10.1007/978-3-030-35723-8_4.

[5] E. Meijering, 'Cell Segmentation: 50 Years Down the Road [Life Sciences]', IEEE Signal Process. Mag., vol. 29, no. 5, pp. 140–145, Sep. 2012, doi: 10.1109/MSP.2012.2204190.

ANNEX I
WORKING WITH ENVIRONMENTS

In this annex the basic workflow to create an environment and to setup your project framework are summarized.

1. Create a *workingfolder* in your directory tree
2. Open a command window, using **cmd**, into your *workingfolder*
3. Create the environment `python -m venv nameofenviro` (if everything is correct a directory *nameofenviro* will be created in you *workingfolder*.
4. Go to *nameofenviro/Scripts* folder
5. Activate the environment `activate.bat`
6. Go back to the *workingfolder* folder using `cd ../..`
7. Copy the *requirements.txt* file into the *workingfolder*
8. Install the libraries using `python -m pip install –r requirements.txt`
9. When the installation of the required libraries is done, you can check it with the command `pip list`

All these steps are summarized in these two figures

